Praktikum Eingebettete Systeme

Praktikumsanleitung

Version 1.6, 13.10.2026

Inhaltsverzeichnis

1. Hinweise zum Praktikumsablauf . . . 1
   1.1. Ablauf . . . 1
   1.2. Bewertung . . . 2
   1.3. Bewertungskriterien der Übungsabgaben . . . 4
2. Coderichtlinien . . . 6
   2.1. Programmierung in Modulen . . . 6
   2.2. Namensgebung . . . 9
   2.3. Dokumentation . . . 11
   2.4. Datentypen . . . 11
3. Entwicklungssystem . . . 12
   3.1. Open429Z-D Entwicklungsboard . . . 12
   3.2. Cortex Mikrocontroller Software Interface Standard (CMSIS) . . . 15
4. STM32F4xx Cube Hardware Abstraction Layer (CubeHAL) . . . 16
   4.1. Wichtige Literaturquellen . . . 17
5. Arbeitsweise mit der CubeIDE . . . 19
   5.1. Import des Workspace-Templates mit Git . . . 19
   5.2. Projektstruktur . . . 19
   5.3. Erstellen der Launch-Configuration . . . 20
   5.4. Kompilieren und Starten einer Anwendung . . . 20
   5.5. Verwendung des Debuggers . . . 21
6. Einführendes Beispiel: LED_on . . . 23
   6.1. Taktversorgung einer Peripherie-Komponente . . . 23
   6.2. Initialisierung eines GPIO-Ports . . . 24
   6.3. Schreiben auf einen GPIO-Port . . . 26
7. Grundlagen . . . 28
   7.1. GPIO, Digitale Ein- und Ausgabe . . . 28
   7.2. Einschub: Umgang mit dem Logic Analyzer . . . 31
   7.3. LCD . . . 36
   7.4. ADC . . . 37
   7.5. DMA . . . 42
   7.6. Timer . . . 45
   7.7. Nested Vector Interrupt Controllers (NVIC) . . . 53
8. Projekt 1: Lüfterregelung . . . 66
   8.1. Ansteuerung von PC-Lüftern . . . 66

9. Projekt 2 69
9.1. Anbindung des BME280-Umgebungssensors über den I^2C-Bus 69
9.2. Kommunikation über CAN 71
10. FAQ 77
11. Literatur 79

Kapitel 1. Hinweise zum Praktikumsablauf

1.1. Ablauf

Vor- und Nachbereitung

Das Praktikum für Eingebettete Systeme wird mit 9 Creditpoints bewertet. Dies entspricht einem Arbeitsaufwand von insgesamt 270 Stunden (9 CP mal 30 Stunden). Die Praktikumstermine dienen der Rücksprache mit den Betreuenden und der Abgabe von Lösungen. Die Aufgaben müssen daher auch außerhalb der Präsenzzeiten bearbeitet werden. Nutzen Sie die Termine, um Fragen zu stellen, insbesondere wenn Sie sich bei der Verwendung der Hardware unsicher sind. Für Fragen außerhalb der Veranstaltungszeiten steht der THM Chat zur Verfügung. Dort können Sie sich gegenseitig helfen oder Unterstützung durch die Betreuenden erhalten.

Gruppeneinteilung und Git-Repositories

Bearbeiten Sie die Aufgaben und Projekte allein oder in Gruppen von maximal zwei Personen. Teilen Sie die Mitglieder Ihrer Gruppe im THM Chat mit, damit die zugehörige GitHub-Gruppe bzw. das Git-Repository bereitgestellt werden kann. Melden Sie sich frühzeitig bei den Betreuenden, wenn es Schwierigkeiten bei der Zusammenarbeit gibt. Bei anhaltenden Problemen kann eine Gruppe aufgelöst werden. Die Aufgaben werden unter Verwendung von Git bearbeitet, damit die Entwicklung nachvollziehbar ist und Feedback möglich bleibt.

Hardwareausleihe und Laborordnung

Jedes Teammitglied erhält ein eigenes Set der für das Praktikum benötigten Hardware. Die Ausgabe muss in der Ausleihliste dokumentiert werden. Jede Person ist für die ihr überlassene Hardware verantwortlich. Fragen Sie die Betreuenden, wenn Sie sich im Umgang mit der Hardware unsicher sind. Lesen Sie vor der Arbeit im Labor die im Moodle-Kurs bereitgestellten Laborordnungen.

Fachgespräch

Die Lösungen werden den Betreuenden am entsprechenden Praktikumstermin (siehe Moodle-Kurs) von den Gruppenmitgliedern vorgeführt und im Fachgespräch erläutert. Zu diesem Zeitpunkt müssen die Programme funktionsfähig und der Code gemäß den Vorgaben in Kapitel 2 dokumentiert sein (Doxygen-Style). Jedes Gruppenmitglied muss die Lösung und den eigenen bzw. gemeinsam erarbeiteten Code erklären können.

Codeabgabe

Nach dem Fachgespräch können Sie Ihren Code noch verbessern und optimieren. Die

finale Codeabgabe muss **bis Sonntag, 23:59 Uhr, nach dem Fachgespräch** im Git-Repository eingecheckt und durch einen **Git-Tag** gekennzeichnet sein (siehe Moodle-Kurs). Der abgegebene Code muss den Vorgaben in Kapitel 2 entsprechen und dokumentiert sein.

PDF-Abgabe

Für jede Abgabe ist eine PDF-Datei zu erstellen, in der die in den Aufgabenblättern *kursiv geschriebenen Fragen* schriftlich beantwortet werden. Die PDF muss im Git-Repository der Gruppe im Verzeichnis `docs/` (im Wurzelverzeichnis des Repositories) eingecheckt werden. Für die PDF-Abgabe gilt **dieselbe Frist wie für den Code: Sonntag, 23:59 Uhr, nach dem Fachgespräch**.

Dokumentation zu den Praktikumsaufgaben

Dokumentieren Sie darüber hinaus Ihre Vorüberlegungen, Entwürfe und Erkenntnisse sowie eine sinnvolle Softwarearchitektur. Hilfreich sind zum Beispiel Komponentendiagramme der verwendeten Treiber und Module oder Zustandsautomaten, die durch Ihre Implementierung umgesetzt werden. Die konkreten Aufgaben und Fragen ergeben sich aus den jeweiligen Aufgabenblättern.

Umgang mit Literatur

Die notwendigen Informationen, um den Umgang mit der hier verwendeten ARM Cortex M4 Mikrocontroller Familie zu erlernen sind bedauerlicherweise über zahlreiche Dokumente verteilt. Diese Anleitung wird Ihnen zu den Teilthemen diskrete Literaturquellen nennen, denen Sie wertvolle Informationen entnehmen können. Ziel ist es, dass sie sich später einmal selbständig in eine neue Mikrocontroller-Famillie einarbeiten können.

Umgang mit dieser Praktikumsanleitung

Im Laufe dieser Anleitung finden Sie immer wieder Code Ausschnitte wie beispielsweise eine Peripherie eingestellt werden kann. Diese Beispiele sind jedoch nicht immer zielführend bei der jeweiligen Aufgabe und zeigen nur wie es aussehen kann, aber nicht muss. Überlegen Sie sich immer genau wie die Peripherie arbeiten soll und wofür genau die einzelnen Konfigurationsmöglichkeiten stehen.

1.2. Bewertung

Die Modulnote setzt sich aus folgenden Bestandteilen zusammen:

- Klausur: **50 %**
- Praktikum: **50 %**

Das Bestehen der Klausur ist Voraussetzung für das Bestehen des Moduls.

Praxisteil

Das Praktikum umfasst **fünf Abgaben**, die innerhalb des Praktikums gleich gewichtet werden. Somit macht jede Abgabe **10 % der Modulnote** aus. Die Bewertung jeder einzelnen Abgabe setzt sich wie folgt zusammen:

- **Fachgespräch: 70 %**
- **Codeabgabe: 25 %**
- **PDF-Abgabe: 5 %**

Code- und PDF-Abgabe werden **gemeinsam pro Gruppe** bewertet, das Fachgespräch dagegen **individuell pro Teilnehmerin bzw. Teilnehmer**. Die Übungen und Projekte werden einem Betreuer vorgeführt und im Fachgespräch abgenommen. Die Vorgaben zu Abgabeformaten, Git-Tag und Frist finden Sie im Abschnitt „Ablauf“ sowie in den jeweiligen Aufgabenblättern.

Fachgespräch

Im Fachgespräch werden die Lösung erläutert und der Lösungsweg diskutiert. Jedes Gruppenmitglied muss einen gleichwertigen Beitrag leisten; die mündliche Leistung wird einzeln bewertet.

Klausur

Am Ende der Veranstaltung erfolgt eine kurze (60 min) Klausur, die zum Bestehen des Moduls bestanden werden muss.

1.3. Bewertungskriterien der Übungsabgaben

Kriterium	Anmerkungen
Programmstruktur / Softwarearchitektur	• Wurde die Funktionalität des Programms sinnvoll auf einzelne Module aufgeteilt? • Sind diese Module wiederverwendbar? • Wurde der Code sauber in Deklaration (Header-Dateien) und Definition (C-Dateien) aufgeteilt? • Werden die Aufgaben des Moduls sinnvoll auf einzelne Funktionen aufgeteilt? • Bildet ein Modul einen in sich abgeschlossenen Funktionsblock? Oder enthält es zu viel oder zu wenig Funktionalität? • Wurde Mehrfach-Inklusion von Header Dateien verhindert? (z.B. durch "Include Guards")  Insbesondere im Main-Modul soll im Normalfall nur die Initialisierung bzw. Deinitialisierung und die Ablaufsteuerung (Hauptschleife) stehen!
Coderichtlinien	• Werden die in Kapitel 2 festgelegten Anforderungen eingehalten?
Code-Dokumentation	• Sind die Code Dateien ordentlich dokumentiert? (Siehe Kapitel 2) • Steht vor jeder Funktion Zweck, Vor-/Nachbedingungen, Beschreibungen von Parametern und Rückgabewert? • Ist korrekt in DoxyGen Style dokumentiert (https://www.doxygen.nl)? • Ist die Dokumentation ausreichend, damit ein anderer Programmierer Ihren Code verwenden oder warten kann?  Vergessen Sie nicht Variablen-Deklarationen, Konstanten und Makros (#define) zu dokumentieren!

Kriterium	**Anmerkungen**
Sauberer Programmierstil	• Ist der Code sauber und einheitlich formatiert? • Ist die Benennung von Funktionen und Variablen sinnvoll und aussagekräftig? • Folgt die Benennung den Vorgaben? (Siehe Kapitel 2) • Wird bei einem Modul sauber zwischen öffentlichen und privaten Funktionen und Variablen unterschieden? • Werden globale Variablen dort, wo es möglich ist, durch statische Variablen mit Getter und Setter Funktionen gekapselt? • Wie wird mit konkurrierenden Zugriffen umgegangen? • Sind alle Variablen, die aus einem Interrupt verändert werden mit `volatile` deklariert? • Sind Interrupt Handler kurz gehalten? • Haben Sie alle 'Warnungen' des Compilers beseitigt?
Fehlerbehandlung	• Werden zur Laufzeit auftretende Fehler und Ausnahmen in geeigneter Weise behandelt? • Werden Abstürze und Dead-Locks durch Fehlerbehandlungen vermieden?
Fachgespräch (Einzelbewertung)	• Werden die Fragen korrekt beantwortet? • Haben Sie sich ggf. tiefergehend mit manchen Aufgaben beschäftigt, als es gefordert war? • Können Codeabschnitte von jedem Teammitglied erläutert werden?

Kapitel 2. Coderichtlinien

2.1. Programmierung in Modulen

Es soll in Funktional gekapselten Modulen entwickelt werden. Ein Modul behandelt eine Zusammenfassung von Funktionalität und besteht aus einer `.c` (c-source) und einer `.h` (c-header) Datei. Dabei enthält die `.h` Datei die Schnittstelle zum Modul, welche durch Datentypen, Definitionen und Funktionsprototypen beschrieben wird. Alle Funktionen, die nicht von außen angesprochen werden sollen, sind **private** Funktionen in der `.c` Datei und müssen als `static` deklariert werden, sodass von außen nicht auf solche zugegriffen werden kann. Globale Variablen dürfen ausschließlich Modul-Global sein, was ebenfalls durch eine `static`-Deklaration bewerkstelligt wird. Ein Beispiel für den Aufbau dieser Dateien ist in Listing 1 und Listing 2 gegeben. Im Projekttemplate finden Sie genauere Darstellungen in den bereits implementierten Modulen.



Die Funktionsweise Ihres Moduls ist leichter verständlich und wartbar, wenn Sie es in logische Bereiche aufteilen. So ist es beispielsweise sinnvoll alles an Hardwarenahem "Bitgeschubse" im unteren Bereich des Moduls zu platzieren (Stichwort: Low-Level Code). Alles Abstraktere, was mit der Funktionsweise des Moduls zu tun hat, könnten Sie in dem oberen Bereich der Implementierung platzieren (Stichwort: High-Level Code). Dadurch kann man sich je nach Problemstellung zielorientiert in der Datei bewegen, auch wenn man das Modul nicht selbst geschrieben hat.

Listing 1. Beispiel für die Struktur einer .c Datei

```
/**
************************************************
* @file        muster.c
* @author      Max Mustermann
* @version     v1.0
* @date        31.03.22
* @brief:      Module for using the periphery xyz
@verbatim
================================================
            ### Resources used ###
    GPIO: PA1-PA5, PG13, PG15
    ADC: ADC1
================================================
            ###     Usage      ###
```

```
    (#) Call 'muster_init()' to initialize the
        module and all the necessary peripheries
    (#) Call 'muster_calc_sum(a, b)' to calculate
        the sum of a and b
    ...
==============================================
@endverbatim
**********************************************
*/

/* Includes */

#include "muster.h"

/* Preprocessor defines */

/* Preprocessor macros */

/* Module intern type definitions */

/* Static module variables */

/* Static module functions (prototypes) */

void init_gpio(void);
void init_adc(void);

/* Public functions */

/**
 *  @brief  Initializes the module and all the necessary periphery
 *  @param  None
 *  @return None
 */
void muster_init(void) {
    init_gpio();
    init_adc();
}

/**
 *  @brief  Calculates the sum of the passed arguments
 *  @param  a   first summand
 *  @param  b   second summand
 *  @return sum of \a and \b
 */
```

```
int16_t muster_calc_sum(int16_t s16_a, int16_t s16_b) {
    int16_t s16_sum = a + b;
    return sum;
}

/* Static module functions (implementation) */

/**
 *  @brief  Initializes the necessary GPIOs for this module
 *  @param  None
 *  @return None
 */
void init_gpio(void) {
    /* Some code for initializing the necessary GPIOs */
}

/**
 *  @brief  Initializes the necessary ADC for this module
 *  @param  None
 *  @return None
 */
void init_adc(void) {
    /* Some code for initializing the necessary ADC */
}
```

Listing 2. Beispiel für die Struktur einer .h Datei

```
/**
**************************************************
* @file        muster.h
* @author      Max Mustermann
* @version     v1.0
* @date        31.03.22
* @brief:      Module for using the periphery xyz
**************************************************
*/

#ifndef MUSTER_MUSTER_H_
#define MUSTER_MUSTER_H_

/* Includes */

#include "stm32f4xx.h"

/* Public Preprocessor defines */
```

```
/* Public Preprocessor macros */

/* Public type definitions */

/* Public functions (prototypes) */

void muster_init(void);
void muster_calc_sum(void);

#endif  /* MUSTER_MUSTER_H_ */
```

2.2. Namensgebung

Alle Namen sind mit underscore zu trennen und klein zu schreiben.



Ein Beispiel wäre: `gueltige_schreibweise`

2.2.1. Benennung von Modulen

Module müssen aussagekräftig benannt werden. Geht es Beispielsweise um ein Modul für die Bereitstellung eines Joysticks, wäre ein passender Name für die Dateien **joystick.c** und **joystick.h**.

2.2.2. Benennung von Variablen

Variablen müssen aussagekräftig benannt werden. Ihr Name muss klein geschrieben werden und als Trennzeichen wird ein underscore verwendet. Der Name beginnt mit dem Datentyp der Variable, falls er zu den Standard-Datentypen gehört. Globale Variablen ist ein `g_` vorzustellen.

Listing 3. Beispiel für Variablennamen

```
/* funktions variablen */
uint8_t  u8_value1;
int32_t  s32_value2;

/* globale variablen */
static uint8_t  g_u8_value_x;
```

2.2.3. Bennenung von Funktionen

Funktionen müssen aussagekräftig benannt werden. Sie haben für die Namensgebung von Funktionen zwischen solchen, die Sie extern bereitstellen und privaten (static) Funktionen zu unterscheiden. Die Namensgebung ist wie bei den Funktions-Variablennamen zu vollziehen. Extern bereitgestellte Funktionen (nicht static / in der .h Datei erwähnt) müssen den Modulnamen als Prefix führen.

Listing 4. Beispiel für Funktionsnamen in dem joystick-Modul

```
/* private Funktionen */
static uint8_t read_pin_states(void);

/* public Funktionen */
uint8_t joystick_get_up_state(void);
uint8_t joystick_get_left_state(void);
```

2.2.4. Preprozessor Makros und Preprozessor Konstanten

Makros und Konstanten müssen aussagekräftige Namen haben. Sie müssen komplett groß geschrieben werden. Zudem muss bei einem Makro oder einer Konstante der Modulname vorangestellt werden.

Listing 5. Beispiel für Preprozessor Makros und Preprozessor Konstanten

```
#define JOYSTICK_UP_GPIO_PIN    GPIO_Pin_9
#define JOYSTICK_UP_GPIO        GPIOA
#define JOYSTICK_IS_UP(x)       ((x) & (1<<9))
```

2.2.5. Datentypen

Wenn Sie eigene Datentypen definieren, schreiben Sie diese klein, trennen die Worte mit einem underscore und fügen ein abschließendes `_t` dem Namen an. Geben Sie aussagekräftige Namen. Ein Datentyp hat mit dem Modulnamen zu beginnen.

Listing 6. Beispiel für einen Datentyp

```
typedef enum {
    EN_UP,
    EN_DOWN,
    EN_LEFT,
    EN_RIGHT,
} joystick_direction_t;
```

2.3. Dokumentation

Dokumentieren Sie im Doxygen Style (https://www.doxygen.nl/). Achten Sie darauf, dass Sie möglichst viele Informationen in Ihre Schnittstellenbeschreibung bringen. Dokumentieren Sie in der `.c` Datei. Im Funktionscode können Sie frei kommentieren. Achten Sie hierbei auf den Richtwert: Je mehr Gedanken Sie sich über einen Abschnitt machen, desto umfangreicher sollten Sie ihn kommentieren. Sparen Sie sich Kommentare, wie "i wird inkrementiert".

2.3.1. Module

Der Modulkommentar muss Informationen, wie Datum der Erstellung, Autoren und eine Beschreibung der Funktionalität des Moduls nach Doxygen-Style beinhalten. Es ist auch eine Beschreibung unterzubringen, wie das Modul verwendet werden soll. Orientieren Sie sich dafür an der Cube HAL. Notieren sie bitte auch, welche Perepherie und Ressourcen (z.B. Timer, ADCs, DMAs, ...) durch ihr Modul belegt werden und damit nicht weiter zur Verfügung stehen! Vergleichen Sie hierfür mit dem Beispiel in Section 2.1.

2.3.2. Funktionen

Erläutern Sie die konkrete Aufgabe der Funktion mit eventuellen Vor- und Nachbedingungen. Beschreiben sie die vollständige Funktionsschnittstelle mit allen Ein- und Ausgabe-Parametern und Rückgabe-Werten. Erläutern sie mögliche Seiteneffekte, die auftreten könnten, und Peripherie, die Sie verwenden.

2.3.3. Variablen

Bei Variablen ist im Kommentar ein gültiger Wertebereich und ggf. Einheiten zu dokumentieren.

2.4. Datentypen

Verwenden Sie ausschließlich Datentypen, die Sie selbst definiert haben oder deren Größe im Namen ersichtlich ist (wie z.B. `uint8_t`). Außerdem dürfen Sie Datentypen der Cube HAL verwenden.

Kapitel 3. Entwicklungssystem

Lernziele:

- Erstes Kennenlernen des Cortex-M4 Mikrocontrollers und des Entwicklungssystems

Der ARM-Cortex-M4 ist eine Prozessorarchitektur der ARM-Cortex-M Familie. ARM steht für "Advanced Risc Machines" und ist der Name eines Unternehmens, das auf die Entwicklung von Prozessorarchitekturen spezialisiert ist. Diese werden an verschiedene Chiphersteller lizenziert und hergestellt. Aufgrund der vielen Lizenznehmer und weiterer Vorteile, wie ein geringer Energieverbrauch, sind ARM-Chips derzeit die meist genutzte Architektur im Bereich der eingebetteten Systeme.

Im Praktikum wird eine Implementierung der Cortex-M4 Architektur der Firma ST-Microelectronics verwendet. Der STM32F429. Dieser Mikrocontroller basiert auf einer 32Bit-Architektur und unterstützt einen Takt von bis zu 180 MHz. Für Programmcode und statische Daten stehen 2 MBytes Flash Speicher zur Verfügung. Weitere 256 kBytes SDRAM stehen für dynamische Daten zur Verfügung. Dazu gibt es weitere 4 kByte SRAM, die an einer separaten Spannung, wie von einer Batterie, als Backup-Speicher dienen. Zur Kommunikation mit PCs und Sensoren gibt es zahlreiche Schnittstellen. Für dieses Praktikum sind vor allem die Interchip-Kommunikation (I²C), eine Serielle Verbindung zum PC (UART) und die Anbindung an einen Feldbus (CAN) interessant.

Tabelle 1. Eigenschaften des STM32F429 Cortex-M4 von ST

Typ	STM32F429
Max. Takt	180 MHz
Flash-ROM	2 MByte
SRAM	256 + 4 kByte
Schnittstellen	USB, I²C, SPI, CAN, USART, Ethernet, …

3.1. Open429Z-D Entwicklungsboard

Sie verwenden im Praktikum das STM32F429 Discovery Board. Auf diesem Board befindet sich ein STLink-V2 SWD Programmer, der dem Programmieren des Mikrocontrollers dient. Neben dem STM32F429ZI Mikrocontrollers befinden sich auf dem Board

- ein TFT LCD mit Touch-Sensor,
- zwei Taster: Reset und User Taster,
- 6 LEDs, wobei nur zwei vom Controller angesteuert werden können,

- ein Mikro-USB Anschluss,
- und ein 64 MBit SDRAM als externer Speicher.

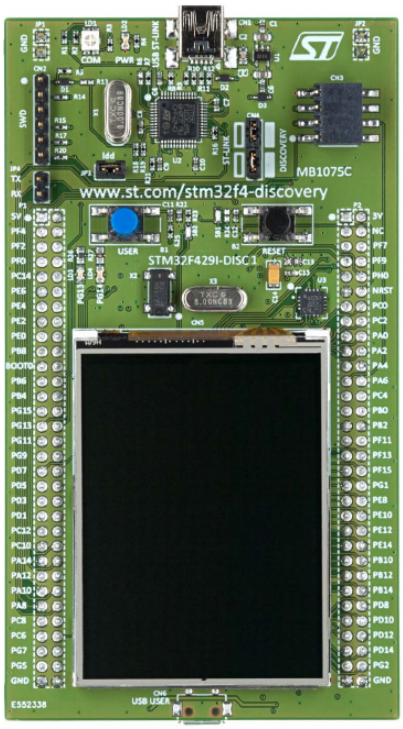


Abbildung 1. STM32F429 Discovery Board, Quelle: [1]

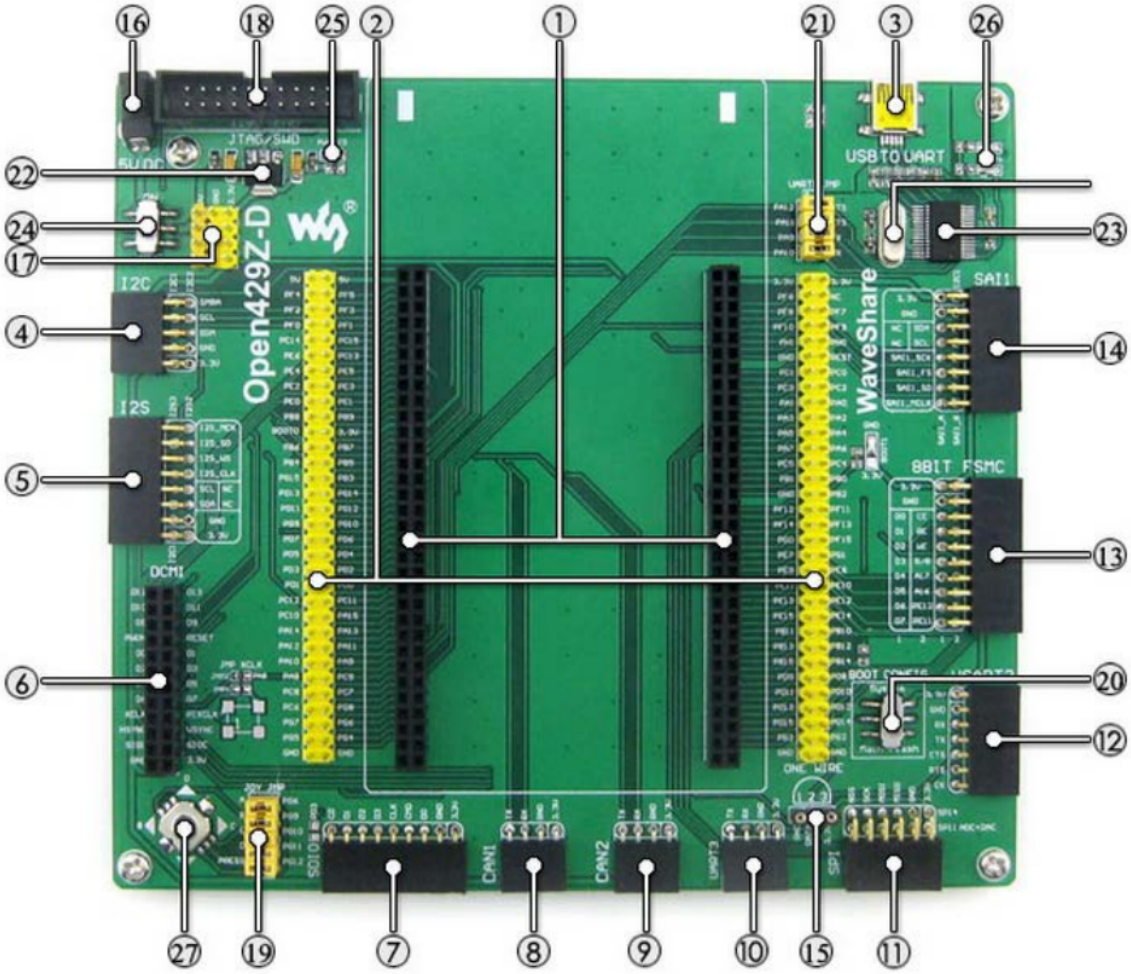


Abbildung 2. Übersichtsbild des verwendeten Basisboards, Quelle: [2]

Auf dem Basis Board haben Sie unter anderem :

1. den Steckplatz für das Discovery Board,
2. alle GPIOs auf eine Stiftleiste geführt, um eine freie Anbindung von Peripherien zu ermöglichen,
3. eine USB Buchse, die angeschlossen an Ihrem PC einen virtuellen COM Port (für eine UART Kommunikation) erzeugt,

11. die Möglichkeit SPI Kommunikationspartner oder das Analog Testboard anzustecken,

17. eine Möglichkeit I²C Kommunikationspartner anzustecken,

27. und einen Joystick.

3.2. Cortex Mikrocontroller Software Interface Standard (CMSIS)

Lernziele:

- Grundlagen zum CMSIS

Auf der Basis des Cortex™-M basierende Mikrocontroller erlangen in der Industrie immer größere Verbreitung. Dies hat zur Folge, dass diese zu einem Industrie-Standard werden. Der CMSIS ist eine herstellerunabhängige Hardwareabstraktionsschicht für die Cortex™-M-Prozessor-Serie. Er realisiert ein einheitliches Konzept für Schnittstellen zu Peripheriefunktionen, Echtzeit-Betriebssystemen und Middleware Komponenten. Dies vereinfacht es entwickelte Software **wiederzuverwenden**, **beschleunigt die Einarbeitungszeit** in neue Mikrocontroller und **verkürzt somit auch die Entwicklungszeit** neuer Geräte. Der CMSIS wurde in enger Zusammenarbeit mit einer Reihe wichtiger Halbleiter- und Softwareanbieter entwickelt, worunter auch Keil und STMicroelectronics waren. In der STM32F4xx CubeHAL wird die CMSIS mitgeliefert und integriert sich, wie unten in Abbildung 3 zu sehen, in die Abstraktionsschichten.

Dabei liefert der CMSIS folgende Abstraktionen, die sich auf den Zugriff des Prozessorkerns und dessen Peripherie beziehen:

- Namensdefinitionen
- Adressdefinitionen
- Hilfsfunktionen für den Zugriff auf Register
- Hilfsfunktionen für den Zugriff auf Peripherie (Quelle: [HPFAFF])

Kapitel 4. STM32F4xx Cube Hardware Abstraction Layer (CubeHAL)

Die **CubeHAL** ist eine von STMicroelectronics entwickelte und kostenlos zur Verfügung gestellte C-Bibliothek. Diese umfangreiche und komfortable Bibliothek bietet dem Programmierer Zugriff auf alle Funktionen der Mikrocontroller-Reihe STM32F4xx. Für die verschiedenen Peripheriekomponenten (Timer, ADC, Interruptcontroller uvm.) stehen jeweils eigene Module zur Verfügung. Die Funktionsweise der einzelnen Module ist immer in den ersten Zeilen der C-Datei des Moduls beschrieben. Dort stehen Informationen zur Konfiguration und Nutzung der Peripherie.

Der Hauptgrund, warum diese Bibliothek hier verwendet wurde ist, dass diese es ermöglicht, mit verhältnismäßig geringen Kenntnissen bezüglich des Prozessors, nach kurzer Zeit funktionsfähige Software zu entwickeln. Dies verkürzt die Einarbeitungszeit und hält so beim Anfänger den Grad der Frustration niedrig. Abbildung 3 in Verbindung mit Abbildung 4 zeigen die Integration der Bibliothek und deren Co-Existenz mit der CMSIS.

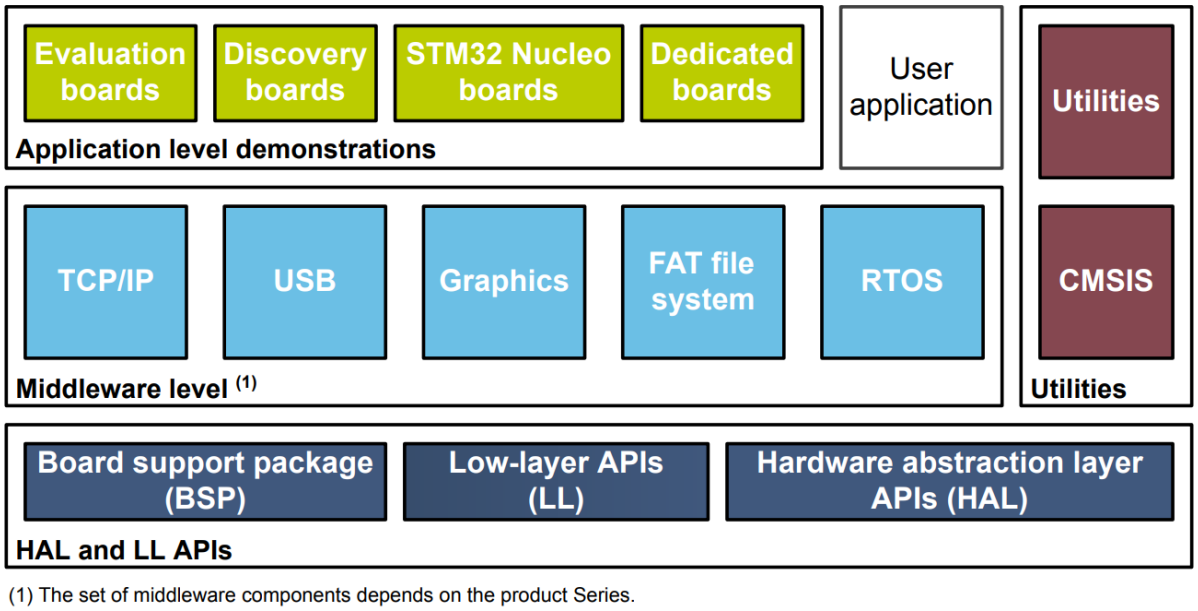


Abbildung 3. Cube Architektur Überblick, Quelle: STM32Cube MCU Package for STM32F4 Series

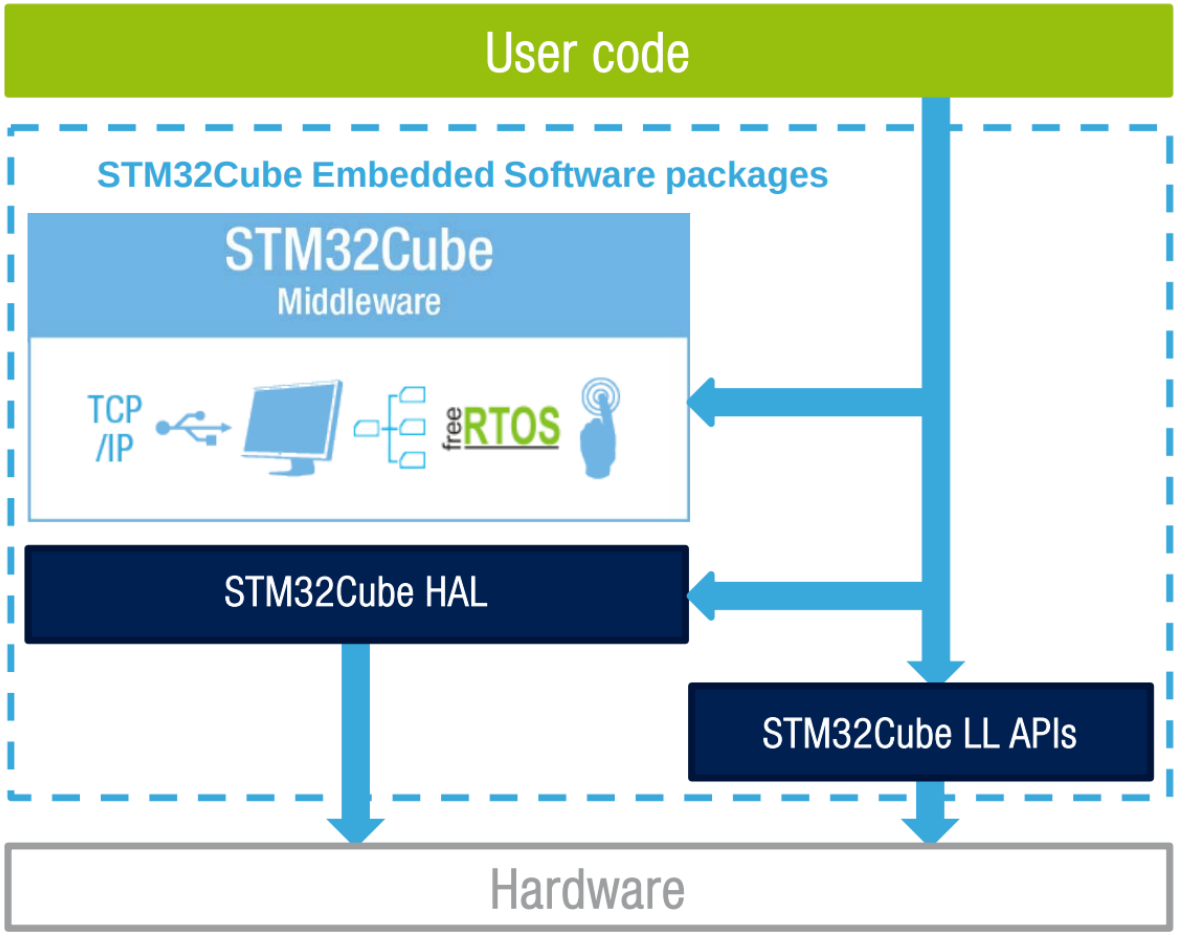


Abbildung 4. CubeHAL Zusammenhänge der Aufrufe, Quelle: STM32 Embedded Software - Overview

4.1. Wichtige Literaturquellen

Um effizient Applikationen für den Cortex-M4 Mikrocontroller erstellen zu können, empfiehlt es sich die folgenden Literaturquellen immer im Zugriff zu haben:

[3] Das ist das sog. Referenz-Manual des Cortex-M4 Controllers, dass eine sehr gute und ausführliche Beschreibung des Controllers und seiner Peripheriekomponenten enthält. Mit diesem Dokument sollten Sie sich immer bei einer neuen Peripherie auseinandersetzen.

[4] Der Schaltplan zum OPEN429Z-D Basis Board.

[5] Der Schaltplan zum 8-Segmente Adapter Board.

[6] Der Schaltplan zum BME280 Adapter Board.

[7] Der Schaltplan zum Analog Test Board.

[1] Das Discovery Kit User Manual, mit Schaltplan.

[2] Das User Manual zum Open429Z-D Basis Board.

[8] Das Datenblatt des STM32F429.

[9] Das Datenblatt zum BME280 Umgebungssensor.

[10] Das Datenblatt zum Gyroskop und Beschleunigungssensor.

[11] Diplomarbeit: Hauke Pfaffelmoser. Diese Arbeit beschreibt neben vielen nützlichen Grundlagen (**achtung, hier ist ein Cortex-M3 in Verwendung**) detailliert das Projekt zum Thema "Lüfterregelung". Darüber hinaus wird hier eine Einführung in das Keil Echtzeitbetriebssystem gegeben. Dies findet in diesem Kurs keine Verwendung, ist jedoch durchaus interessant.



Die exakten Literaturquellen finden Sie im Literaturverzeichnis am Ende dieses Dokuments. Die Dokumente selbst finden Sie im Git-Repository, wo Sie auch diese Praktikumsanleitung gefunden haben (Link zum Repository im Moodle-Kurs).



Beantworten Sie nun am besten die Fragen im Kapitel **Entwicklungssystem (CMSIS, Cube)** im ersten Aufgabenblatt `abgabe1.pdf`

Kapitel 5. Arbeitsweise mit der CubeIDE

Lernziele:

- Arbeitsweise mit Git innerhalb der CubeIDE
- Editieren von Programmen in der CubeIDE
- Kompilieren und Flashen eines Programms
- Starten und Debuggen

In diesem Kapitel lernen Sie im Rahmen einer Schritt-für-Schritt-Anleitung, wie Sie die Entwicklungsumgebung bedienen und ein erstes Projekt zum Laufen bekommen. Sie erfahren, wie Sie eine Anwendung editieren, kompilieren und debuggen können.

5.1. Import des Workspace-Templates mit Git

In diesem Kurs arbeiten Sie mit Git, indem Sie das von uns vorgegebene Template-Workspace benutzen. Wie genau das Template-Workspace mithilfe von Git innerhalb der CubeIDE importiert werden kann, ist in diesem Video erklärt. Das Video ist ebenfalls im Moodle-Kurs zu finden.

5.2. Projektstruktur

In dem importierten Template-Workspace finden Sie das Projekt **00_Introduction**, mit dessen Unterordner-Struktur Sie sich vertraut machen sollten. Der Ordner `HAL_Driver` enthält alle Peripherie-Treiber der HAL und der LL, von denen Sie viele innerhalb dieses Kurses benutzen und sich in den `.c`-Dateien der Treiber über deren genaue Verwendung informieren werden müssen. Im Unterordner `src` befindet sich der eigentliche Quellcode ihres Projektes in Form von der Datei `main.c`. Können Sie sich vorstellen, was die darin enthaltenen Funktionsaufrufe bewirken? Betrachten Sie dafür auch den Inhalt des Ordners `modules`.

Vergessen Sie nicht, den Dokumentationskopf zu Beginn der `main.c` zu ergänzen bzw. zu aktualisieren. Tragen Sie dort Ihren Namen ein und ergänzen Sie auch die restlichen Informationen, falls nötig.

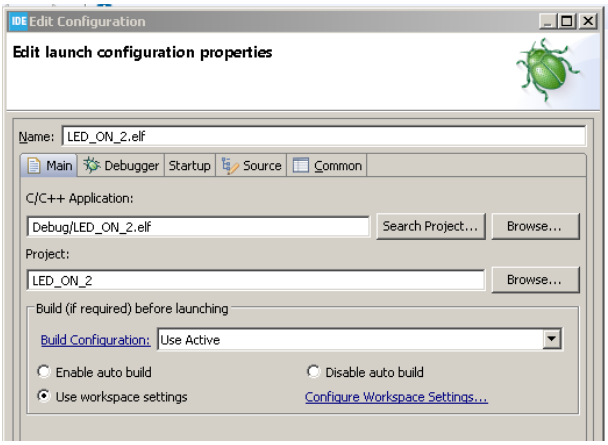
Wenn Sie neue Funktionen erstellen, denken Sie daran diese mit einem Dokumentationskopf zu versehen, im Regelfall geschieht dies immer vor der Funktions-Deklaration in der `.h`-Datei. Halten Sie sich insgesamt an die in Kapitel 2 beschriebenen Richtlinien.

5.3. Erstellen der Launch-Configuration

Eine Start-Konfiguration (Launch-Configuration) erfüllt die Folgenden Aufgaben:

- Auswahl der Werkzeugzusammenstellung für die Zielplattform, bei uns STM-32 MCU C/C++.
- Einstellung der Debug-Probe (ST-Link V2), also des Gerätes, welches das fertige (kompilierte und gelinkte) HEX-File auf unseren Controller (Target) überträgt, sowie ein Debugging des Targets ermöglicht.

Um die Launch-Configuration zu erstellen, klicken Sie rechts auf ihr Projekt und wählen Sie im Kontextmenü **Debug As→STM32 MCU C/C++ Application**. Dafür muss selbstverständlich der STM32 schon über die ST-Link-Schnittstelle an Ihren Entwicklungsrechner angeschlossen sein.



In dem erscheinenden Dialogfenster müssen Sie normalerweise nichts abändern und können die Konfiguration mit **OK** bestätigen.

5.4. Kompilieren und Starten einer Anwendung

Nach dem Bestätigen der Launch-Configuration wird automatisch das Projekt kompiliert, gebunden und ausgeführt. Dadurch werden die folgenden Schritte ausgeführt:

- Kompilieren und Binden.
- Übertragen (Flashen) des Hex-Files auf den Mikrocontroller (Target)
- Start des Debuggers und Wechsel in die Debug-Perspektive.

Ist eine Launch-Configuration einmal erstellt, so kann ein erneutes Kompilieren und Debuggen des Projektes auch durch das Wählen von **Run→Debug** im Menü oder das Drücken des grünen Käfers in der Icon-Leiste der CubeIDE gestartet werden.

Wenn Sie ihr Projekt nur auf Fehler prüfen möchten, genügt es natürlich auch **Project→Build Project** oder den Hammer in der Icon-Leiste zu betätigen.



Eine ausführliche Anleitung zur Verwendung der Entwicklungsumgebung finden Sie in der Programm-Hilfe und im Internet.

5.5. Verwendung des Debuggers



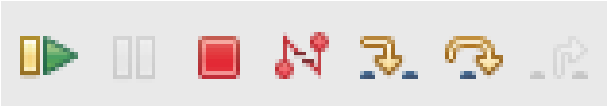
Die Verwendung des Debuggers wird in der Hilfe der Cube-IDE sehr schön und detailliert beschrieben unter: **Help→Help Contents** und dort unter **C/C++ Development User Guide > Tasks > Running and debugging projects**. Lesen Sie diesen Abschnitt bitte aufmerksam durch, es erleichtert ihnen später die Entwicklungsarbeit sehr!

Zum Kompilieren, Binden und Ausführen ihres Projektes wählen Sie im Menü **Run→Debug** oder drücken Sie die Taste **F11**. Anschließend wechselt die IDE in den Debugger-Mode, sofern ihr Code keine syntaktischen Fehler enthält.

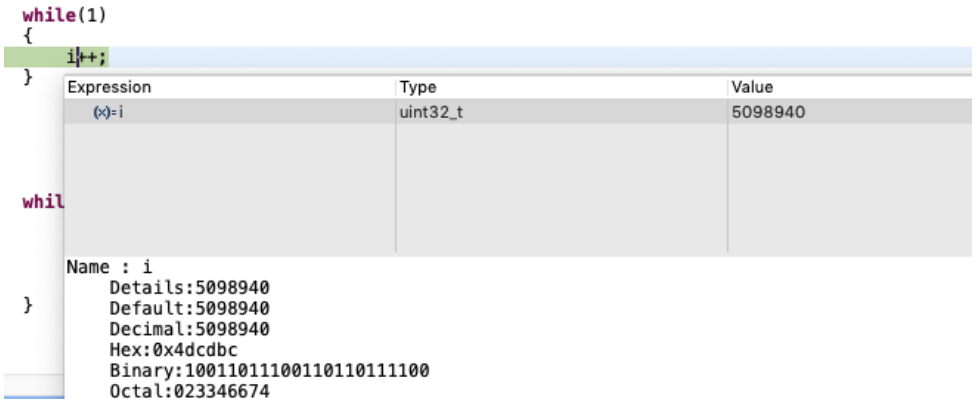
In dieser Ansicht können Sie ihr Programm im Ganzen oder schrittweise abarbeiten und sich Variablen oder Registerinhalte anzeigen lassen.

Nach Aufruf des Debuggers steht der Program-Cursor am Anfang der **main()**-Funktion.

Über die Icon-Leiste haben Sie die Möglichkeit ihr Programm zu starten (Play-Button unten im Bild oder **F8**), zu pausieren (Pause-Button), zu stoppen (Stop-Button oder **Strg** + **F2**) oder in den diversen Einzelschritt-Varianten durchzugehen. Der Einzelschritt-Mode **Step-Into** (fünftes Symbol unten im Bild oder **F5**) springt auch in den Quellcode von aufgerufenen Funktionen, während **Step-Over** (sechstes Symbol oder **F6**) Funktionsaufrufe im Ganzen als einen Schritt ausführt. Möchte man den Rest der Funktion, in der man sich gerade befindet bis zu ihrem Ende ausführen und zurück zum Aufrufer springen, so benutzen Sie **Step-Return** (siebtes Symbol oder **F7**).



Nutzen Sie die Möglichkeit Break-Points (Haltepunkte) zu setzen. Wurde die Ausführung angehalten, so können Sie sich u.A. den Inhalt von Variablen anschauen, indem Sie den Mauscursor darüber setzen.



Ein weiteres wichtiges Debugger-Tool stellen die **"Special function registers"** (SFRs) dar, welche Sie im Debugger-Modus als eigenen Tab im rechten Teil des Bildschirms finden. Hier können Sie sich konkrete Register bestimmter Peripherien, die Sie später kennenlernen werden, ansehen und dadurch oft Fehler in Ihrer Software aufdecken. Auch weitere Werkzeuge des Debuggers wie **Live Expressions** könnten für Sie später interessant werden. Erfahren Sie in der Dokumentation der IDE mehr darüber und allgemein über die Benutzung des Debuggers.



Beantworten Sie nun am besten die Fragen im Kapitel **Arbeitsweise mit der CubeIDE** im ersten Aufgabenblatt `abgabe1.pdf`

Kapitel 6. Einführendes Beispiel: LED_on

Lernziele:

- Umgang mit Ressourcen
- Aktivieren von Peripherie-Taktquellen
- Initialisierung von Peripherie-Komponenten (in diesem Fall GPIO-Ports)
- Schreiben auf einen GPIO-Port

Sie wissen nun wie Sie ein Programm editieren, übersetzen, auf den Mikrocontroller übertragen und debuggen können. Jetzt lernen Sie den Umgang mit digitalen Ausgängen kennen, indem Sie in dem Projekt **00_Introduction** eine Anwendung erstellen, die die beiden LEDs auf dem Entwicklungsboard in Betrieb setzt. Darüber hinaus lernen Sie die wichtigsten Dokumente kennen, die Ihnen den Umgang mit dem Cortex-M4 Mikrocontroller erleichtern.

Zum Ausgeben und Einlesen von externen Signalen verfügt unser Controller über viele kleine Kontaktbeinchen (Pins), die innerhalb so genannter General Purpose IO-Ports zu Gruppen à 16 Pins zusammengefasst werden. Bei einem GPIO-Port handelt es sich um eine Peripherie-Komponente, die über die Funktionen der STM32Cube HAL konfiguriert und angesteuert werden kann.

Um die Information, an welchen GPIO-Pins die beiden anzusteuernden LEDs des Discovery-Boards angeschlossen sind, zu beschaffen, muss Ihnen die Bedeutung der drei Datenblätter **STM32F4-Reference-Manual.pdf**, **STM32F429-Datasheet.pdf** und **discovery-kit_user-manual.pdf** klar werden. Das Reference Manual stellt Informationen zu der Benutzung von Speicher und interner Peripherien der STM32F4-Familie dar, während das Datasheet einen spezielleren funktionalen Überblick über den STM32F427- und den in diesem Kurs verwendeten STM32F429-Microcontroller bietet.

Das User-Manual des Discovery-Boards beschreibt schließlich die auf der Platine befindlichen zusätzlichen externen Peripherien wie die zwei Buttons, den L3GD20-Gyroskopen oder die von Ihnen in diesem Kapitel anzusteuernden LEDs. Betrachten Sie Tabelle 6 dieses Manuals, so finden Sie in der Spalte **LED** die Information, dass die grüne LED des Discovery-Boards unter anderem an den Pin PG13 angeschlossen ist.

6.1. Taktversorgung einer Peripherie-Komponente

Alle intern genutzten Peripherie-Komponenten müssen zunächst mit einem Takt (Clock) versorgt werden, bevor Sie verwendet werden können. Das hat u.A. den Hintergrund, dass

die Peripherie eigene Register hat. Diese Register benötigen einen Takt, um ihre Werte zu aktualisieren. Dieses Konzept haben Sie bereits in der Veranstaltung Digitaltechnik kennengelernt. Wenn Sie also versuchen einen Peripheriebaustein zu konfigurieren, ohne den Takt aktiviert zu haben, werden diese Werte nicht in die notwendigen Register übernommen.

Bevor wir also einen GPIO-Port benutzen können, müssen wir zunächst dafür sorgen, dass dieser mit einem Takt versorgt und dadurch aktiviert wird. Dies erreicht man am Beispiel des zu benutzenden Pins PG13 (grüne LED) durch Aufruf des Makros:

```
__HAL_RCC_GPIOG_CLK_ENABLE();
```

RCC steht für Reset and Clock Control und bezeichnet in der STM32Cube HAL die Gruppe der Peripherie-Komponenten, die für die Konfiguration der Taktversorgung zuständig ist. Hier wollen wir GPIO-Port G mit einem Takt versorgen. Das Makro `__HAL_RCC_GPIOG_CLK_ENABLE()` aktiviert den Takt, während `__HAL_RCC_GPIOG_CLK_DISABLE()` ihn deaktivieren würde.

6.2. Initialisierung eines GPIO-Ports

Nachdem wir nun den Takt (Clock) des GPIO-Ports G aktiviert haben, können wir damit fortfahren den Port zu konfigurieren.

Jeder Pin eines Ports kann als Eingabe- (Input) oder Ausgabe-Pin (Output) konfiguriert bzw. initialisiert werden. Da wir eine LED einschalten wollen, müssen wir den entsprechenden Pin 13 unseres Ports G als Ausgabe-Pin einstellen.



Neben der Konfiguration für digitale Ein-/Ausgangssignale unterstützt jeder Port-Pin viele weitere Modi, z.B. zum Einlesen analoger Signale.

Um einen GPIO-Port zu initialisieren, verwenden wir den Funktionsaufruf:

```
HAL_GPIO_Init(GPIOG, &gpio_init);
```

Das Präfix **HAL_GPIO_...** deutet wieder darauf hin, dass es sich um eine Funktion im direkten Zusammenhang mit der Peripherie-Komponente General Purpose IO des Controllers handelt. Der erste Parameter bezeichnet wieder den GPIO-Port G. Der zweite Parameter ist diesmal die Speicheradresse einer Initialisierungsstruktur, die folgendes Format besitzt:

```
typedef struct
{
    uint32_t Pin;
    uint32_t Mode;
    uint32_t Pull;
    uint32_t Speed;
    uint32_t Alternate;
} GPIO_InitTypeDef;
```



In der CubeIDE lässt sich diese Struktur genauer begutachten. Dafür kann mit der rechten Maustaste auf den Strukturnamen geklickt werden und anschließend auf **Open Declaration (F3)**. Dies ist auch für jegliche Funktion möglich.

Bevor diese Struktur ausgefüllt und übergeben werden kann, muss sie jedoch zuerst einmal angelegt werden. Dies geschieht durch folgende Variablendeklaration:

```
GPIO_InitTypeDef gpio_init;
```

Anschließend kann die Struktur ausgefüllt werden:

```
gpio_init.Pin = GPIO_PIN_13;            ①
gpio_init.Mode = GPIO_MODE_OUTPUT_PP;   ②
gpio_init.Pull = GPIO_NOPULL;           ③
gpio_init.Speed = GPIO_SPEED_MEDIUM;    ④
```

① Legt fest, welche Pins initialisiert werden sollen. Mittels `GPIO_PIN_x` (wobei x für die Nummer des Pins steht) kann ein einzelner, mit `GPIO_PIN_All` können alle und durch eine bitweise Oder-Verknüpfung auch mehrere Pins (`GPIO_PIN_1 | GPIO_PIN_3` wählt Pins 1 und 3 aus) zur Initialisierung ausgewählt werden.

② Legt den Modus des Pins fest (Ein-/Ausgabe, digital/analog, usw.). `GPIO_MODE_OUTPUT_PP` stellt den Port als digitalen Ausgang mit Push-/Pulltreiber ein. Um einen Pin als digitalen Eingang zu konfigurieren, könnte hier z.B. `GPIO_MODE_INPUT` stehen.

③ Legt fest, ob der Pull-Up- oder Pull-Down-Widerstand der ausgewählten Pins aktiviert wird. Mit `GPIO_NOPULL` wird kein Widerstand aktiviert. Über `GPIO_PULLUP` oder `GPIO_PULLDOWN` lässt sich entsprechend der Widerstand aktivieren.

④ Legt die Geschwindigkeit (Flankensteilheit) des Pin-Treibers fest. Eine zu hohe Geschwindigkeit kann in ungünstigen Fällen zu Fehlern durch Leitungsreflexionen führen und unter anderem Auswirkungen auf die Energie haben, weshalb man hier

die Möglichkeit hat den Wert bei Bedarf herab zu setzen.

Hier noch einmal ein komplettes Beispiel zur Konfiguration von Pin PG13 für die Ansteuerung der grünen LED:

```
__HAL_RCC_GPIOG_CLK_ENABLE();           ①

GPIO_InitTypeDef gpio_init;             ②

gpio_init.Pin = GPIO_PIN_13;            ③
gpio_init.Mode = GPIO_MODE_OUTPUT_PP;   ③
gpio_init.Pull = GPIO_NOPULL;           ③
gpio_init.Speed = GPIO_SPEED_MEDIUM;    ③

HAL_GPIO_Init(GPIOG, &gpio_init);       ④
```

① Aktivierung des Taktes von GPIO Port G

② Anlegen der Konfigurations-Struktur

③ Konfiguration von PG13 zur Ansteuerung der grünen LED

④ Port initialisieren

6.3. Schreiben auf einen GPIO-Port

Die STM32Cube HAL bietet eine Funktion, um Pins eines GPIO-Ports zu setzen oder zu löschen. Zusätzlich lässt sich über eine weitere Funktion ein Pin togglen.

Listing 7. Schreiben eines GPIO-Ports.

```
// Pin x des Ports y setzen
HAL_GPIO_WritePin(GPIOy, GPIO_PIN_x, GPIO_PIN_SET);
// Pin x des Ports y löschen
HAL_GPIO_WritePin(GPIOy, GPIO_PIN_x, GPIO_PIN_RESET);
// Pin x des Ports y togglen
HAL_GPIO_TogglePin(GPIOy, GPIO_PIN_x);

/* Wobei x die Pinnummern 0-15 einnehmen und y die Portbezeichnungen A-G. */
```

Hierzu einige Beispiele bezüglich Listing 7.

Listing 8. Beispiele zum Setzen, Löschen und Togglen von Pins eines GPIO-Ports.

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_1, GPIO_PIN_SET);                ①
```

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_1 | GPIO_PIN_5, GPIO_PIN_RESET);  ②
HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_All);                            ③
```

① Setzen des Pin 1 von GPIO Port D.

② Löschen der Pins 1 und 5 von GPIO Port A.

③ Togglen aller Pins von GPIO Port B.



Eine genaue Beschreibung der Peripherie selbst finden Sie im **Reference Manual** der verwendeten Controller Familie. Informieren Sie sich immer zuerst über das Reference Manual, wenn Sie sich in eine neue Peripherie einarbeiten. Im Anschluss, nachdem Sie diese verstanden haben, sollten Sie erst das Modul aus der STM32CubeHAL zu der entsprechenden Peripherie in Augenschein nehmen.



Informationen zu den Funktionen / der Cube HAL API einer Prozessor-Peripherie finden Sie in der Datei `stm32f4xx_hal_<peripherie>.c / .h`. In der C-Datei finden Sie zu Beginn der Datei immer eine ausführliche Beschreibung, wie das Modul zu verwenden ist. In der `.h` Datei finden Sie u.A. alle Strukturdefinitionen mit Erläuterungen zu den Elementen und Verweisen auf mögliche Werte, welche die einzelnen Elemente einer Struktur annehmen dürfen. Die Funktionen und deren genaue Schnittstellen Beschreibung finden Sie ebenfalls in der C-Datei.



Bearbeiten Sie nun am besten die Aufgaben im Kapitel **LED_on** im ersten Aufgabenblatt `abgabe1.pdf`

Kapitel 7. Grundlagen

Während die ersten Schritte noch eng begleitet waren, sind Sie jetzt an der Reihe. In diesem Kapitel befassen wir uns zunächst mit den grundlegenden Funktionen des ARM-Cortex M4, um dann in den nächsten beiden Kapiteln vollständige Projekte umzusetzen.

7.1. GPIO, Digitale Ein- und Ausgabe

7.1.1. Digitale Ausgabe: Waveshare 8-Segment-Board

Zwar wurde Ihnen in Kapitel 6 schon die Ansteuerung von GPIO-Ausgängen vorgestellt, nun befassen Sie sich allerdings mit einem etwas komplexeren Baustein zur Anwendung dieser Controller-Peripherie. Sie werden das 8-Segment-Board der Firma Waveshare in Betrieb nehmen, um schließlich an den einzelnen Positionen Ziffern anzeigen zu können.

Lernziele:

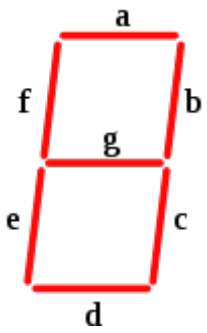
- Lesen von Schaltplänen
- Funktionsweise des 8-Segment-Boards
- Saubere Programmstruktur mittels Modularisierung (*.h- und *.c-Dateien)
- Festigung der Konfiguration von GPIO-Outputs
- Ansteuerung des 8-Segment-Boards

Funktionsweise des 8-Segment-Boards

Die Ansteuerung des 8-Segment-Boards geschieht einerseits über die vier Leitungen **CNTL1**, **CNTL2**, **CNTL3**, **CNTL4**, welche die einzelnen Ziffern mit Strom versorgen. Die acht einzelnen Segmente einer Ziffer werden andererseits mithilfe der Leitungen **a**, **b**, **c**, **d**, **e**, **f**, **g** und **dot** angesteuert, wobei diese Bezeichnungen der allgemeinen Benennung der Segmente entsprechen (siehe Bild unten). Die Leitung **point** für den Doppelpunkt des Displays verhält sich ähnlich wie die der acht Segmente, wobei **CNTL2** und **CNTL3** jeweils für die Stromversorgung einer Hälfte des Doppelpunkts verantwortlich sind.



Diese acht (bzw. neun, wenn man **point** dazuzählt) Segment-Leitungen sind low-aktiv, sodass man einen Low-Pegel anlegen muss, um das Segment zu aktivieren!



Wichtig zu beachten ist hier, dass sich die vier Ziffern diese acht Segment-Leitungen teilen. Wäre also beispielsweise die erste und dritte Ziffer aktiv (High-Pegel auf Leitungen CNTL1 und CNTL3) und es liegt ein Low-Pegel an den Leitungen a, b, g, e und d an, so würde man an **beiden** Positionen die Ziffer **2** sehen.

Diese Einsparung von Segment-Leitungen für jede der vier Ziffern hat zur Folge, dass es nicht möglich ist, **gleichzeitig** an zwei verschiedenen Positionen zwei unterschiedliche Ziffern anzuzeigen. Aufgrund der Trägheit unseres Auges gibt es allerdings trotzdem eine Möglichkeit, dieses gewünschte Verhalten zu erreichen. Fällt Ihnen die Lösung ein?

Anschluss des 8-Segment-Boards

Suchen Sie sich am Waveshare-Board eine der schwarzen Buchsen aus, an welche Sie das 8-Segment-Ansteckboard anschließen möchten.



Von der Anzahl der Steckverbindungen ergibt sich nur eine der zur Verfügung stehenden Buchsen.

Nun stellt sich die Frage, an welche konkreten GPIO-Pins des Microcontrollers die Anschlüsse dieser Buchse geleitet werden und somit auch welche GPIO-Pins zur Ansteuerung des 8-Segment-Boards benötigt werden. Dafür werden der Schaltplan des Waveshare-Boards (**Open429Z-D-Schematic.pdf** unter Ressourcen) und der des 8-Segment-Boards (**8-SEG-LED-Board-Schematic.pdf**) benötigt. Suchen Sie sich im Ersteren den Ihrer ausgewählten Buchse zugewiesenen Block heraus und gleichen Sie diesen mit dem 10X2-Header aus dem Schaltplan des 8-Segment-Boards ab.



Die beiden Header-Blöcke sind zueinander an der x-Achse gespiegelt! Orientieren Sie sich also an den Pin-Nummern 1 bis 20!



Bearbeiten Sie nun am besten die Aufgaben im Kapitel **GPIO: Digitale Ausgabe** im ersten Aufgabenblatt `abgabe1.pdf`

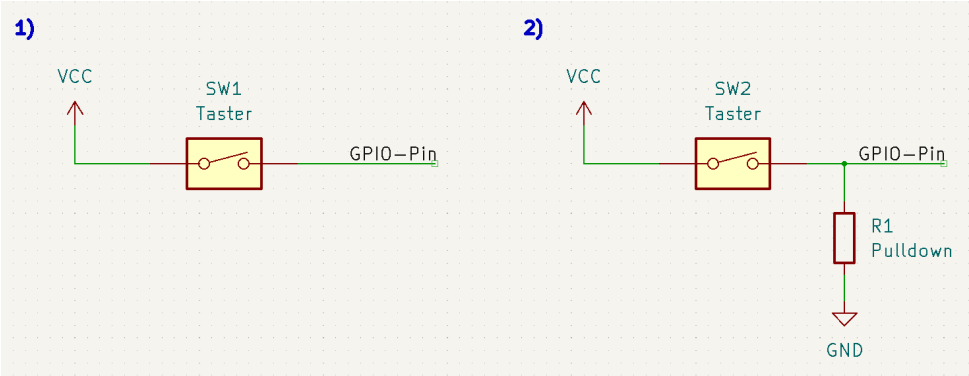
7.1.2. Joystick

Lernziele:

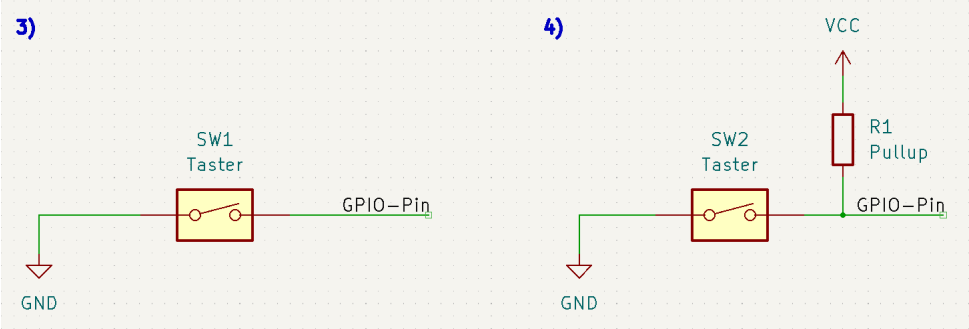
- Funktionsweise von Tastern
- Konfiguration von GPIO-Inputs
- Auslesen von Digitaleingängen
- Ansteuerung des Onboard-Joysticks

Funktionsweise von Tastern

Taster eignen sich als Baustein dafür, eine Eingabe durch den Benutzer zu registrieren. Eine Möglichkeit ist es, den Eingang des Mikrocontrollers beim Tastendruck mit dem High-Pegel zu verbinden (Schaltung 1 unten im Bild). Dann spricht man von einem sogenannten *high-active*-Eingang, bei dem die gewollte Aktion durch einen High-Pegel ausgelöst wird. Es ergibt sich allerdings das Problem, dass der Pegel am Eingang bei geöffnetem Schalter undefiniert ist. Umgangen wird diese Problematik meist durch einen zusätzlichen **Pull-down-Widerstand** am GPIO-Eingang des Mikrocontrollers (Schaltung 2). Nun ist der Eingang bei beiden Tasterzuständen definiert (High-Pegel bei geschlossenem, Low-Pegel bei offenem Taster).



Analog dazu kann ein Taster auch einen *low-active*-Eingang am Mikrocontroller ergeben, wenn dieser beim Schließen des Tasters auf GND gezogen wird (Schaltung 3). Auch hier entsteht die Problematik eines undefinierten Zustandes bei offenem Taster, welches hier mit einem **Pullup-Widerstand** gelöst wird (Schaltung 4).



In Section 6.2 haben Sie schon erfahren, dass unser Mikrocontroller die Möglichkeit bietet, GPIO-Pins einen internen Pullup- bzw. Pulldown-Widerstand zuzuschalten und so weitere externe Bauteile einzusparen.

Waveshare-Onboard-Joystick

In der letzten Aufgabe haben sie gelernt, wie man unter der Verwendung der STM32Cube HAL einen GPIO-Pin für die digitale Ausgabe eines Signals konfiguriert. Nun geht es darum die Position des auf dem **Waveshare-Open429Z-D-Boards** befindlichen Joysticks mithilfe der fünf dafür verantwortlichen GPIO-Pins für die digitale Eingabe einzulesen. Der Joystick kann in acht unterschiedliche Richtungen betätigt werden. Einerseits in die gewohnten Richtungen oben, unten, links und rechts (jede mit einer der Leitungen **A**, **B**, **C** oder **D** verbunden), andererseits aber auch in die vier möglichen diagonalen Richtungen. Diese haben allerdings keine eigenen Leitungen, sodass z.B. bei einem Drücken nach obenrechts sowohl die Leitung für oben, als auch die für rechts aktiv wäre. Außerdem kann der schwarze Taster auch in die Mitte runtergedrückt werden, welches durch einen Pegel auf der Leitung **PRESS** gekennzeichnet wird.

Das Vorgehen beim Einlesen eines Digitaleingangs ähnelt weitgehend dem setzen eines Digitalausgangs. Zunächst muss die Clock des entsprechenden GPIO-Ports aktiviert werden, um diesen in Betrieb zu setzen. Als nächstes erfolgt die Konfiguration der gewünschten Port-Pins durch Erstellen und Ausfüllen der schon kennengelernten Initialisierungs-Struktur.



Bearbeiten Sie nun am besten die Aufgaben im Kapitel **GPIO: Digitale Eingabe** im ersten Aufgabenblatt `abgabe1.pdf`

7.2. Einschub: Umgang mit dem Logic Analyzer

7.2.1. Installation von PulseView

Für die Visualisierung der vom Logic Analyzer empfangenen Signale muss zunächst eine passende Software installiert werden. **PulseView** ist das zur Open-Source-Software **Sigrok** gehörige Frontend und funktioniert mit dem Logic Analyzer von AZ Delivery, der Teil des

Hardware-Pakets für das Praktikum ist.

Die eigentliche Installation gestaltet sich für die üblichen Betriebsysteme vergleichsweise einfach: Es existieren Installationsdateien zum Download, die alle notwendige Software enthalten.

Bis auf die Treiber für den eigentlichen Logic Analyzer, dieser muss unter Umständen manuell nachinstalliert werden.

7.2.2. Installation der Treiber

Windows:

Unter Windows muss der von Sigrok genutzte libusb-Treiber für den angeschlossenen Logic Analyzer separat installiert werden. Hierzu wird bei jeder PulseView-Installation ein Tool namens Zadig mitinstalliert, welches über den entsprechenden Startmenü-Eintrag gestartet werden kann.

Linux:

Im Normalfall funktioniert eine mit entsprechenden Rechten installiertes PulseView ohne zusätzliche Treiberinstallation, in einigen Fällen kann es allerdings notwendig sein, Firmware-Dateien manuell nachzuinstallieren. Bei Fehlern mit der Geräteerkennung bzw. dem Zugriff sollte die hier erhältliche Firmware heruntergeladen werden.

Alle enthaltenden .fw-Dateien müssen in das Verzeichnis `/usr/local/share/sigrok-firmware` entpackt werden. Nun sollte PulseView problemlos mit dem Logic Analyzer kommunizieren können.

7.2.3. Anschließen des Logic Analyzers an die Hardware

Um ein Signal aufzuzeichnen, muss der Logic-Analyzer an die Signalquelle angeschlossen werden.

Hierzu wird zunächst ein Channel-Pin des Logic Analyzer mit dem entsprechenden Pin auf dem Waveshare Board, oder dem Discovery-Board selber, und daraufhin einer der GND Pins mit einem GND Pin auf dem Waveshare Board verbunden. Beide Verbindungen werden mit den beigelegten Jumperkabeln hergestellt.

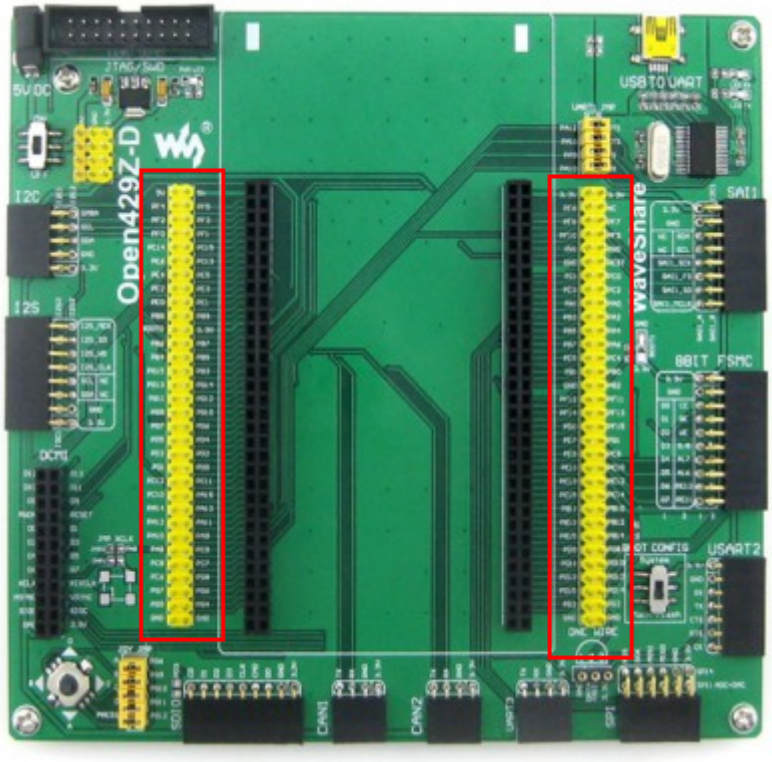


Abbildung 5. Waveshare Board Pinleisten

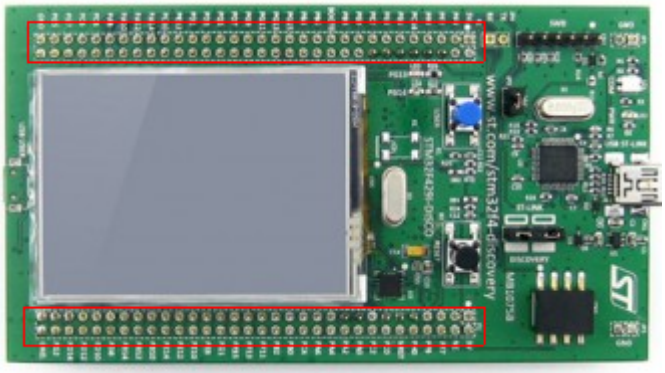


Abbildung 6. Discovery Board Pinleisten

Nachdem die Anschlüsse hergestellt und der Logic Analyzer mit dem PC verbunden ist, kann PulseView gestartet und die Messung begonnen werden. Sicherheitshalber sollte überprüft werden, ob der Microcontroller mit der korrekten Software läuft.

7.2.4. Periodenlänge und Duty Cycle messen

Zusammen mit PulseView ermöglicht die Nutzung des Logic Analyzers die Messung der tatsächlichen Periodenlänge und des Duty Cycle eines digitalen Signals.

Nach dem Anschließen des Logic Analyzers und dem Starten von PulseView sollten zunächst die Einstellungen überprüft werden.



Abbildung 7. PulseView Toolbar

Wurde ein Logic Analyzer erkannt, wird das voreingestellte Demo Device durch den erkannten ersetzt, im Beispiel durch **Saleae Logic**. Wird trotz der durchgeführten Schritte in den Installationsabschnitten nach wie vor das Demo Device angezeigt, so kann der echte Logic Analyzer über das Dropdown Menu ausgewählt werden.

Je nach Signal muss vor dem Starten der Messung auch die Abtastfrequenz eingestellt werden. Die in Abbildung 7 gezeigten Einstellungen sollten im Zweifelsfall aber genügen. Hierbei nicht vergessen, die Menge der abgetasteten Samples mit der Frequenz zu erhöhen, da sonst pro Lauf nicht genug Messungen vorgenommen werden.

Nach der Konfiguration kann die Messung mit dem Run-Button gestartet und wieder beendet werden. Ist die Messung abgeschlossen, kann man das Signal entweder manuell mit dem Messwerkzeug - in Abbildung 7 links mit einem roten Kreis markiert - oder mit den bereitgestellten Signaldekodern - mit einem roten Rechteck markiert - betrachten.

Für ein PWM-Signal wählt man nach Druck auf den Dekoder-Button PWM aus und erhält neben den angezeigten Signalen im Hauptfenster eine weitere Zeile.

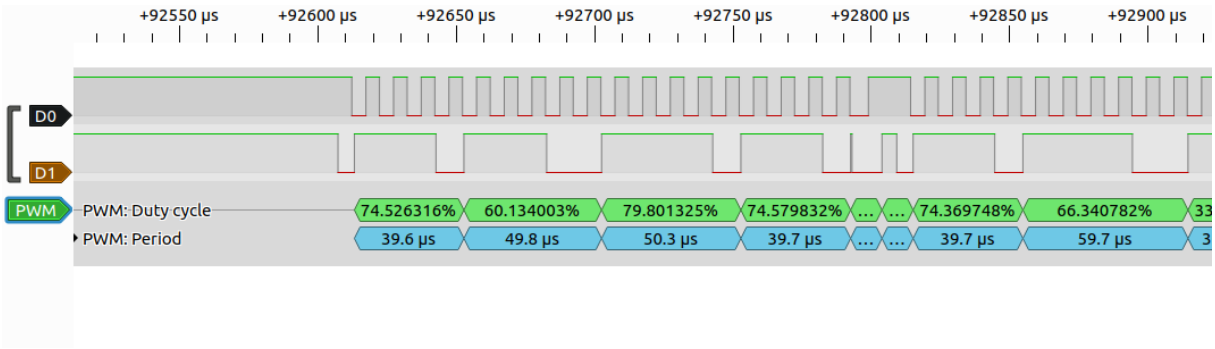


Abbildung 8. PWM Decoder

Damit diese Zeile die Werte für Duty Cycle und Period ausrechnet und anzeigt, muss zunächst der gewünschte Eingangskanal ausgewählt werden. Hierzu genügt ein Klick auf den linken, mit "PWM" beschrifteten Pfeil und die Auswahl des Kanals aus dem Dropdown Menu - im Beispiel D1.

7.2.5. Kommunikationsprotokolle dekodieren

Dank der mitgelieferten Protokolldecoder kann PulseView empfangene Signale auch nach verschiedenen Protokollen dekodieren. Dies kann nützlich sein, um bspw. I²C- oder SPI-Signale zu überprüfen.

Die Konfiguration geschieht analog zum PWM-Beispiel, allerdings muss vorab darauf

geachtet werden, dass alle notwendigen Signale vom Logic Analyzer eingelesen werden - im folgenen Beispiel sind dies SDA und SCL der I²C-Verbindung.

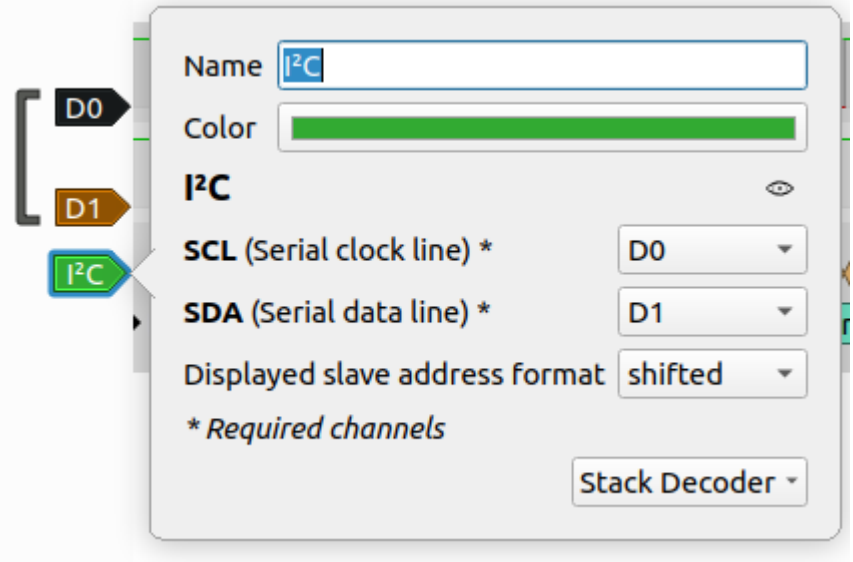


Abbildung 9. I²C Konfiguration

Bei Kommunikationsprotokollen erhält man nicht Duty Cycle und Period, sondern Read/Write Bits und Operationsbeschreibung. Auf diese Weise lässt sich die Implementierung verschiedener Kommunikationsschnittstellen auf Korrektheit prüfen.

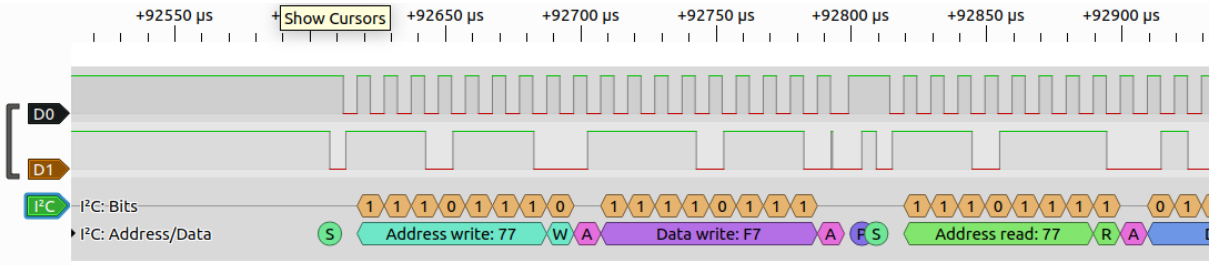


Abbildung 10. I²C Decoder

7.2.6. Troubleshooting

Sollten trotz Befolgen der Anleitung Probleme auftreten, können folgende Hinweise Abhilfe schaffen:

- Anschluss des GND Pin überprüfen
- Anschlüsse der Signalkanäle überprüfen
- Möglichst andere USB-Geräte entfernen
- Auf mehrere geöffnete Instanzen von PulseView prüfen und überzählige Instanzen ggf. schließen
- Die Abtastfrequenz niedriger einstellen

- Alle anderen Programme schließen

Sollten danach immer noch Probleme bestehen, lohnt sich ein Nachschlagen im User Manual oder den FAQ, außerdem stehen natürlich auch Tutoren und Dozenten jederzeit via Element Chat zu Verfügung.

7.3. LCD

Lernziele:

- Einbindung einer weiteren Bibliothek
- Aktivierung und Nutzung des Displays
- Ausgeben von Variablen mittels sprintf(...)-Funktion

Nachdem Sie nun wissen, wie Sie Benutzereingaben mittels Joystick entgegen nehmen können, wollen wir uns um eine weitere sehr bequeme Möglichkeit kümmern mit dem Benutzer zu interagieren: Das grafische LC-Display.

Das Display verfügt über eine Auflösung von 240 x 320 Pixeln bei einer Farbtiefe von 18 Bit (262144 Farben).

7.3.1. Einbinden der LCD-Lib

Bevor Sie das Display nutzen können, müssen Sie die zugehörigen Funktions-Prototypen einbinden. Dazu stellt die Bibliothek die Datei "lcd.h" bereit.

Die notwendigen Dateien sind bereits im Template enthalten. In den Programmmodulen, in denen Sie das Display verwenden möchten, inkludieren Sie lediglich die "lcd.h" Header-Datei.

Eine Schnittstellendokumentation der einzelnen LCD-Funktionen finden Sie in den Kommentaren der *.c Datei.

7.3.2. Konfigurieren des Displays

Die Initialisierung und Konfiguration des Display ist einfach.

Durch Aufruf von `lcd_init()` kann das Display initialisiert werden. Standardmäßig ist damit die Hintergrundfarbe auf weiß eingestellt und das Display hat eine vertikale Position.



Die Bibliothek "lcd.h" bedient sich der komplexeren Bibiotheken "ILI9341_STM32_Driver.h", für die grundlegenden Funktionen, und

"ILI9341_GFX.h", für kleine grafische Anwendungen. Durch einbinden der "lcd.h" sind diese automatisch mit eingebunden und deren Funktionalitäten können verwendet werden. Die wichtigsten Funktionalitäten sind jedoch mit "lcd.h" abstrahiert.



Bevor Sie ab hier in der Praktikumsanleitung fortfahren, sollten Sie die Aufgaben im Kapitel **LCD** im ersten Aufgabenblatt `abgabe1.pdf` bearbeiten und somit die erste Abgabe abschließen.

7.4. ADC

Lernziele:

- Grundlegendes Verständnis des Analog-Digital-Converters (ADC)
- Konfiguration des ADCs (ADC-Takt, Sample-Time, Kalibrierung)
- Interpretation des Messergebnisses

Ein Analog-Digital-Wandler (englisch Analog-Digital-Converter, kurz ADC) ist ein elektronisches Bauelement zur Umsetzung analoger Eingangssignale in einen digitalen Datenstrom, der dann weiterverarbeitet oder gespeichert werden kann. Während mithilfe einer Sample-and-Hold-Schaltung die analoge Eingangsspannung konstant gehalten und abgetastet werden kann, erfolgt daraufhin die Quantisierung des Signals (eigentliche Umwandlung in digitalen Wert). Es existieren unterschiedliche Verfahren zur Umsetzung dieser Quantisierung, wobei die *sukzessive Approximation* weit verbreitet ist und auch von den ADCs des Cortex-M4 Mikrocontrollers umgesetzt wird. Hierbei wird die zu digitalisierende Eingangsspannung mit einer durch einen DAC (Digital-Analog-Converter, Gegenstück zum ADC) erzeugten Referenzspannung verglichen. Das Ergebnis dieses Vergleichs bestimmt das höchstwertige Bit des digitalen Wertes. Durch Anpassung der Referenzspannung und Wiederholung des Vergleichs wird die Referenzspannung immer mehr (*sukzessiv*) der Eingangsspannung angenähert (*Approximation*) und so die restlichen Bits des gewandelten digitalen Werts bestimmt.

Der Cortex-M4 Mikrocontroller hat je nach Ausführung bis zu drei 12-Bit ADCs, die mittels eines schnellen Analogmultiplexers (schneller Umschalter) bis zu 19 analoge Quellen (davon 16 externe, 2 interne und V_{BAT}) erfassen können. Externe Quellen können beliebige analoge Signale (z.B. von Sensoren) sein, die von außen zugeführt werden. Bei den internen Quellen handelt es sich um eine Batteriespannungsüberwachung und einen internen Temperatursensor.

Die Konvertierung eines analogen Wertes kann einzeln angestoßen werden (*Single Conversion Mode*) oder kontinuierlich (*Continuous Conversion Mode*) erfolgen. Soll mit einem ADC

mehr als eine Quelle in kurzer zeitlicher Abfolge erfasst werden, bietet sich die Verwendung des sog. *Scanning Conversion Modes* an. In diesem Modus wird ein ADC jeweils nach einer erfolgten Konvertierung auf eine neue Quelle (Kanal) geroutet. Auf diese Weise können quasi-simultan bis zu 18 Kanäle erfasst werden. Dabei ist ein externer Kanal in der Regel fest mit einem bestimmten Pin am Gehäuse des Controllers verbunden und repräsentiert eine bestimmte Quelle wie z.B. ein analoges Potentiometer.



Für genauere Informationen lesen Sie Kapitel 13 im Reference Manual.

7.4.1. GPIO Konfiguration für analoge Signale

Der erste Schritt, bevor Sie den Analog-Digital-Converter verwenden können, ist wie immer die Konfiguration eines GPIO-Pins. Dieses Mal für das Einlesen analoger Signale (`GPIO_MODE_ANALOG`). Ein Pull-Up- bzw. Pull-Down-Widerstand wird hier nicht benötigt.



Vergessen Sie nicht, die Taktversorgung für die zu konfigurierende GPIO Peripherie zu aktivieren!

7.4.2. Konfiguration des ADCs

Wie bei jeder anderen Peripherie auch, muss zunächst der Takt des benutzten ADCs aktiviert werden. Handelt es sich um ADC1, so lautet der Aufruf `__HAL_RCC_ADC1_CLK_ENABLE()`. Zur Konfiguration eines ADCs mithilfe der HAL muss wie immer zunächst eine Initialisierungs-Datenstruktur ausgefüllt werden, in diesem Fall befindet sich diese in einer Variable vom Typ `ADC_HandleTypeDef`. Im Rahmen der CubeHAL wird der Großteil der Peripherie mit sogenannten Handles referenziert.



Für jede genutzte Instanz einer Peripherie (mit Ausnahme der GPIOs) muss es solch ein Handle geben. Dieses stellt die Verknüpfung zwischen Peripherie und Konfiguration dar. Im späteren Verlauf werden diese Handles wichtig, wenn es darum geht in einem Interrupt Handler zu identifizieren, welche Peripherie-Instanz den zu behandelnden Interrupt geworfen hat. Zu diesem Thema wird es genauere Informationen geben, wenn die Interrupts behandelt werden.

Das Anlegen und Ausfüllen dieser Struktur könnte wie folgt aussehen:

```
ADC_HandleTypeDef ADC_handle_structure;

ADC_handle_structure.Instance = ADC1;                                    ①

ADC_handle_structure.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;     ②
```

```
ADC_handle_structure.Init.Resolution = ADC_RESOLUTION_12B;                ③
ADC_handle_structure.Init.DataAlign = ADC_DATAALIGN_RIGHT;                ④
ADC_handle_structure.Init.ScanConvMode = DISABLE;                         ⑤
ADC_handle_structure.Init.EOCSelection = ADC_EOC_SINGLE_CONV;             ⑥
ADC_handle_structure.Init.ContinuousConvMode = ENABLE;                    ⑦
ADC_handle_structure.Init.NbrOfConversion = 1;                            ⑧
ADC_handle_structure.Init.ExternalTrigConv = ADC_SOFTWARE_START;          ⑨
```

① Legt fest, welcher der drei ADCs konfiguriert wird (ADC1 - ADC3). Hier wird also die ADC Instanz bestimmt.

② Die ADCs sind an den Peripheriebus APB2 angeschlossen, sodass sie mit einem Takt von 84 MHz beliefert werden. Da sie aber nur mit einer maximalen Frequenz von 36 MHz fehlerfrei funktionieren (vgl. Data Sheet, S.149), müssen die 84 MHz mithilfe dieses Parameters runtergeteilt werden. Da er nur die Werte 2, 4, 6 oder 8 annehmen kann, ergibt sich für den Teiler der Wert 4, da

$$\frac{84MHz}{4} = 21MHz < 36MHz$$

③ Dieser Parameter gibt an, wie hoch die Auflösung des ADCs sein soll bzw. wie viele Bits für die Darstellung des digitalen Ergebnisses einer Wandlung zur Verfügung stehen.

④ Dieser Parameter gibt an, wie das Ergebnis einer Wandlung innerhalb des 16-Bit breiten Ergebniswortes ausgerichtet werden soll. Hier wird in der Regel `ADC_DATAALIGN_RIGHT` verwendet.

⑤ Kann entweder aus- (`DISABLE`), oder eingeschaltet (`ENABLE`) werden. Wird diese Option eingeschaltet, lässt sich der ADC so konfigurieren, dass er alternierend mehrere analoge Eingangskanäle "abscannt".

⑥ Spezifiziert, ob das EOC-Flag (End Of Conversion) nach der Wandlung jeder Quelle oder der gesamten Kanal-Sequenz (im Scan Mode) gesetzt wird.

⑦ Der Continuous Conversion Mode kann entweder aus- oder eingeschaltet werden. Wird der Modus aktiviert, wird der ADC nach Abschluss einer Wandlung automatisch erneut gestartet und stellt somit kontinuierlich neue Ergebnisse bereit. Schaltet man diesen Modus aus, so muss jede Wandlung explizit durch ein Kommando angestoßen werden und das Ergebnis abgewartet werden.

⑧ Legt im Scan Conversion Mode fest, wie viele Quellen die zu wandelnde Sequenz enthält.

⑨ Hier hat man die Möglichkeit, ein externes Signal als Trigger zum Anstoßen einer Wandlung zu verwenden. Ein solches Signal könnte von einer externen Leitung (z.B. einem Taster) oder intern durch einen der Timer des Controllers ausgelöst werden. `ADC_SOFTWARE_START` legt allerdings fest, dass keine externen oder internen Signale Beachtung finden, sondern die Wandlung manuell im Code gestartet wird.



An die hier beschriebenen (und weiterführenden) Information kommen Sie, wenn Sie sich die .c- und .h-Dateien der CubeHAL für die ADC Peripherie anschauen. Dazu wurde schon erklärt, wie man die Dateien erkennt und welche Informationen sie enthalten. Außerdem ist die gleichzeitige Betrachtung des oben genannten Kapitels im Reference Manual zum Verständnis notwendig.

Durch Aufruf der Funktion `HAL_ADC_Init(&ADC_handle_structure)`, welcher man einen Adresse auf die vorher erstellte Handle-Datenstruktur übergibt, wird die ADC-Basiseinheit grundlegend initialisiert. Es müssen aber auch noch die gewünschten Eingangskanäle (Channels) konfiguriert werden. Auch hierfür muss eine Datenstruktur erstellt und gefüllt werden (vom Typ `ADC_ChannelConfTypeDef`). Im folgenden Beispiel wird Channel 10 konfiguriert:

```
ADC_ChannelConfTypeDef ADC_channel_structure;

ADC_channel_structure.Channel = ADC_CHANNEL_10;                 ①
ADC_channel_structure.Rank = 1;                                 ②
ADC_channel_structure.SamplingTime = ADC_SAMPLETIME_84CYCLES;   ③
```

① Definiert den zu konfigurierenden Kanal, wobei `ADC_CHANNEL_0` bis `ADC_CHANNEl_15` externe Signale repräsentieren, an `ADC_CHANNEL_17` V_{REFINT} und an `ADC_CHANNEL_18` V_{BAT} und der interne Temperatursensor angeschlossen sind. Diese Information finden Sie im Reference Manual, Abschnitt 13.3.3.

② Repräsentiert den Rang des Channels. Wird ein ADC im Scanning-Mode betrieben, in welchem er mehrere Kanäle im Wechsel digitalisiert, kann über den Rang eines Channels dessen Position in der "Scanning-Liste" eingestellt werden.

③ Konfiguriert die Sample- and Hold-Einheit des verwendeten Kanals und ist für das Verständnis des ADCs grundlegend wichtig. `ADC_SAMPLETIME_84CYCLES` ist im Prinzip eine Zeitangabe, die in ADC-Takten ausgedrückt wird (in diesem Fall 84 Takte). Unser ADC wird im obigen Beispiel mit 21 MHz betrieben, sodass man hier also eine Sample-Time von

$$\frac{84}{21MHz} = 4\mu s$$

einstellt. Im Detail bedeutet das, dass jeder Abtastwert (Sample) das Integral des analogen Eingangswertes (Mittelwert) über einen Zeitraum von 4 µs beschreibt. Änderungen des Eingangssignals, die innerhalb dieses kurzen Zeitraums liegen, können also vom ADC zeitlich nicht mehr aufgelöst bzw. erfasst werden.

Die Konfiguration des ADCs wird durch den Aufruf von `HAL_ADC_ConfigChannel(&ADC_handle_structure, &ADC_channel_structure)` abgeschlossen, um die Konfiguration des vorher

angelegten Channels durchzuführen.

7.4.3. Starten der Wandlung

Um den ADC nun endgültig in Betrieb zu nehmen, ist ein einziger weiterer Schritt nötig. Der ADC befindet sich nach Konfiguration im Standby-Modus und muss durch den Aufruf von `HAL_ADC_Start(&ADC_handle_structure)` aufgeweckt werden. Ist der ADC im Continuous Mode konfiguriert, so muss diese Funktion einmalig aufgerufen werden, da der ADC nun kontinuierlich weiterläuft und fleißig immer wieder neue Werte aufnimmt.

Um einen Abtastwert auszulesen, muss lediglich das Ende einer Wandlung abgewartet und anschließend das Datenregister des ADC ausgelesen werden. Dies geschieht durch:

```
ret = HAL_ADC_PollForConversion(&ADC_handle_structure, 1000);
if(ret != HAL_TIMEOUT)
{
    uint32_t ADC_val = HAL_ADC_GetValue(&ADC_handle_structure);
    /* ... */
}
```

Der zweite Parameter des Aufrufs `HAL_ADC_PollForConversion(⋯)` stellt einen Timeout in Millisekunden dar, nach dessen Ablaufen die Funktion das Warten auf den Abschluss der Wandlung beendet und `HAL_TIMEOUT` zurückgibt.



Prüfen Sie **immer** die Rückgabewerte der HAL, wenn sie bereitgestellt werden. Hierdurch können Sie auf Fehlverhalten reagieren um kritische Fehler zu verhindern.



Trotz Continuous Mode kann es vorkommen, dass der ADC nach einer Konvertierung anhält und jeder weitere Aufruf von `HAL_ADC_PollForConversion()` zu einem Timeout führt. Dies liegt an der sogenannten "Overrun Detection" des ADC und tritt immer dann auf, wenn `HAL_ADC_PollForConversion()` nicht schnell genug aufgerufen wird bevor der nächste Konvertierungsvorgang startet. Die HAL bietet leider keine einfache Möglichkeit damit umzugehen, als Workaround kann jedoch vor jedem Aufruf von `HAL_ADC_PollForConversion()` der ADC neu gestartet werden. Mehr Informationen zur Overrun Detection des ADCs finden Sie im Reference Manual in Kapitel 13.8.

7.4.4. Interpretation der Abtastwerte

Die Abtastwerte des ADCs werden im vorzeichenlosen 32-Bit Format (`uint32_t`) zurückgegeben, wobei der Wertebereich jedoch lediglich die im Attribut `Resolution` definierte Anzahl an Bits beträgt. Beträgt diese beispielsweise 12 Bits, so werden nur Werte im Bereich 0 - 4095 zurückgegeben.

Der Maximalwert 4095 entspricht hierbei der extern am Controller angeschlossenen ADC-Referenzspannung, die beim hier verwendeten Board 3,3 Volt entspricht. Ein ADC-Wert kann somit (falls nötig) mittels folgender Formel in eine analoge Spannung umgerechnet werden:

$$\frac{3.3V \cdot ADC_val}{4095}$$



Wenn Sie diese Formel in einem Programm einsetzen, achten Sie darauf, dass Sie keine Rundungsfehler aufgrund der Ganzzahl-Datentypen generieren. Hier bietet es sich an, einfach die Einheit zu wechseln und statt 3.3V 3300mV einzusetzen, um das Ergebnis in handlichen Millivolt zu erhalten.



Bearbeiten Sie nun am besten die Aufgaben im Kapitel **ADC** im zweiten Aufgabenblatt `abgabe2.pdf`

7.5. DMA

Die Technik des direkten Speicherzugriffs erlaubt integrierten Peripheriekomponenten wie ADCs oder Kommunikations-Schnittstellen ohne den Umweg der CPU über das interne Bussystem direkt zu kommunizieren. Eine Kommunikation kann in drei Richtungen erfolgen:

- Peripherie nach Arbeitsspeicher
- Arbeitsspeicher nach Peripherie
- Arbeitsspeicher nach Arbeitsspeicher

Die Vorteile dieser Technik sind eine schnelle Datenübertragung bei gleichzeitiger Entlastung des Prozessors. Unser Mikrocontroller verfügt über zwei DMA-Controller, die jeweils 8 **Streams** zur Verfügung stellen, welche alle einzeln konfiguriert und ihnen Prioritäten zugewiesen werden können. Jeder dieser Streams ist mit einem **DMA Request** verbunden, welcher aus 8 möglichen **Channels** ausgesucht werden kann. Jeder Channel wiederum ist einer bestimmten Peripherie (Timer, SPI, I2C, ADC, etc.) zugewiesen, die einen Request auf diesem Channel auslösen kann. Diese Zuordnung von DMA-Channel zu Peripheriekompo-

nente wird als **DMA Request Mapping** bezeichnet. Ein Request startet eine Datenübertragung, die in Ihrer Art und Weise konfiguriert werden kann. Wie solch eine Konfiguration aussehen kann, werden Sie in den folgenden Abschnitten erfahren.



Lesen Sie Kapitel 10 "DMA Controller (DMA)" im Reference Manual für genauere Informationen zur Funktionsweise des DMA Controllers.

7.5.1. Konfiguration des DMA-Controllers

Zunächst muss, wie bei den zuvor beschriebenen Peripheriekomponenten, der gewünschte DMA-Controller mit einem Takt versorgt werden. Finden Sie dieses Mal selber heraus, wie Sie dies bewerkstelligen können. Anschließend kann die Initialisierungsstruktur `DMA_HandleTypeDef` ausgefüllt werden, mit deren Hilfe ein DMA-Channel für eine Übertragung konfiguriert und initialisiert werden kann. Dieses Ausfüllen könnte so aussehen:

```
DMA_HandleTypeDef DMA_handle_structure;

DMA_handle_structure.Instance = DMA1_Stream6;                              ①

DMA_handle_structure.Init.Channel = DMA_CHANNEL_7;                         ②
DMA_handle_structure.Init.Direction = DMA_MEMORY_TO_PERIPH;                ③
DMA_handle_structure.Init.PeriphInc = DMA_PINC_DISABLE;                    ④
DMA_handle_structure.Init.MemInc = DMA_MINC_ENABLE;                        ⑤
DMA_handle_structure.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;   ⑥
DMA_handle_structure.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;      ⑦
DMA_handle_structure.Init.Mode = DMA_CIRCULAR;                             ⑧
DMA_handle_structure.Init.Priority = DMA_PRIORITY_HIGH;                    ⑨
DMA_handle_structure.Init.FIFOMode = DMA_FIFOMODE_DISABLE;                 ⑩

HAL_DMA_Init(&DMA_handle_structure);                                       ⑪
```

① Beschreibt, welcher der beiden DMAs und welcher Stream konfiguriert werden sollen.

② Definiert, welcher Channel des ausgewählten Streams konfiguriert werden soll.

③ Über dieses Feld legt man fest, in welche der drei oben genannten Richtungen der DMA-Datentransfer geschehen soll.

④ Bestimmt, ob die Adresse der Peripherie nach einem Datentransfer inkrementiert werden soll oder nicht.

⑤ Bestimmt, ob die Adresse im Speicher nach einem Datentransfer inkrementiert werden soll. Ein möglicher Anwendungsfall wäre hier beispielsweise das Speichern von Wandlungsergebnissen eines ADCs in einem Array. So würde der Arrayindex nach jeder Wandlung und jedem DMA-Transfer automatisch erhöht werden.

⑥ Hiermit wird angegeben, ob das Datenregister der Peripheriekomponenten 8, 16 oder 32 Bit breit ist.

⑦ Hiermit wird angegeben, ob der Platz im Speicher von dem bzw. zu dem transportiert werden soll 8, 16 oder 32 Bit groß ist.

⑧ Beschreibt, in welchem Modus der DMA Stream betrieben werden soll. Neben dem normalen Modus ist vor allem `DMA_CIRCULAR` interessant, da damit die Datenübertragung zirkular geschehen kann, also im Speicher wieder beim Start beginnt, wenn das Ende eines Pakets erreicht ist.

⑨ Für die Priorisierung beim Nutzen von mehreren DMA-Streams kann eines der folgenden vier Prioritäts-Leveln festgelegt werden:

- `DMA_PRIORITY_LOW`
- `DMA_PRIORITY_MEDIUM`
- `DMA_PRIORITY_HIGH`
- `DMA_PRIORITY_VERY_HIGH`

⑩ Hier kann festgelegt werden, ob der DMA-Stream im "Direct Mode" oder im "FIFO Mode" betrieben werden soll. Beim ersteren verursacht ein DMA Request den sofortigen Transfer vom bzw. zum Speicher, beim letzteren wiederum werden die Daten in einem First-In-First-Out-Puffer zwischengespeichert, bis dieser zu einem gewissen Schwellenwert gefüllt ist.

⑪ Durch diesen Aufruf wird schließlich der DMA-Stream mit der vorher gefüllten Datenstruktur initialisiert.

Ein DMA-Stream stellt Ihnen verschiedene Optionen zur Durchführung einer Übertragung bereit. Ein typischer Fall ist, dass Daten von einer Peripheriekomponente, wie z.B. dem ADC, ausgelesen und in den Speicher übertragen werden. Ziel der Übertragung kann z.B. eine 16-Bit Variable sein, deren Wert aktualisiert wird oder auch ein Datenarray, in den nacheinander mehrere Werte abgelegt werden. Auch die umgekehrte Übertragungsrichtung vom Speicher zur Peripherie ist kein seltener Fall und dient z.B. zur Ausgabe eines Datenpuffers auf einer Schnittstelle.

7.5.2. Auslösen eines DMA-Requests

Die Konfiguration eines DMA-Channels löst noch keine Übertragung aus. Dies erfordert die Anforderung eines DMA-Requests auf dem Channel. Ein solcher Request kann manuell durch Ihr Programm ausgelöst werden. Meist ist aber gewünscht, dass der Request durch eine an der Übertragung beteiligte Peripheriekomponente ausgelöst wird. Beispielsweise soll der DMA-Controller nach erfolgreicher Durchführung einer ADC-Wandlung den Inhalt des ADC Datenregisters automatisch in den Speicher übertragen, ohne die CPU damit zu belästigen.

Um dies zu erreichen, muss die Konfiguration dieser Peripheriekomponente so angepasst werden, dass sie automatisch einen DMA-Request auslöst. Am Beispiel vom ADC wären das folgenden Anpassungen:

1. Bei der ADC-Konfigurations-Datenstruktur muss die Komponente `.DMA_Handle` mit einem Zeiger auf die oben erläuterte DMA-Datenstruktur gefüllt werden.
2. Betreibt man den DMA-Stream im zirkularen Modus und den ADC im "Continuous Mode", so muss auch die Komponente `.Init.DMAContinuousRequests` mit `ENABLE` gefüllt werden.
3. Statt dem Aufruf `HAL_ADC_Start(&ADC_handle_structure)` muss der ADC mit der Funktion `HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef* hadc, uint32_t* pData, uint32_t Length)` gestartet werden. Dabei beschreibt der zweite Parameter die Adresse des Speicherplatzes, an dem das Ergebnis der ADC-Wandlung durch den DMA abgelegt werden soll und `Length` die Anzahl der Datenpakete, die vom ADC zum Speicher transportiert werden sollen.



Die Reihenfolge, in der Sie einzelne Komponenten initialisieren und starten, hat einen direkten Einfluss darauf, ob sie so wie geplant funktionieren. Hier zum Beispiel müsste zuerst der DMA-Stream initialisiert und dann der ADC konfiguriert und gestartet werden, da `HAL_ADC_Start_DMA()` auch den DMA startet und dies nicht vor dessen Initialisierung passieren darf!



Bevor Sie ab hier in der Praktikumsanleitung fortfahren, sollten Sie die Aufgaben im Kapitel **DMA** im zweiten Aufgabenblatt `abgabe2.pdf` bearbeiten und somit die zweite Abgabe abschließen.

7.6. Timer

Lernziele:

- Grundlegendes Verständnis der Cortex-M4 Timer
- Verständnis der Output-Compare-Einheit
- Erzeugung von Pulsweiten-modulierten Signalen

Häufig müssen Mikrocontroller Abläufe in einer exakten zeitlichen Abfolge ausführen (Ampelsteuerung) oder den Zeitpunkt auftretender Ereignisse bestimmen (Geschwindigkeitsmessung). Exaktes Timing ist bei Mikrocontrollern häufig das A und O!

Darüber hinaus sind bei einem Mikrocontroller viele interne Timings erforderlich, z.B. zur Steuerung einer DMA-Übertragung oder zur Taktgenerierung für Kommunikationsschnitt-

stellen.

Realisiert werden Zeitgeber und Zeitmesser mittels getakteter Zählwerke, der sog. Timer. Der Cortex-M4 verfügt insgesamt über 14 unabhängige Timer. Jeder Timer basiert auf einem 16-Bit-Zähler (Timer 2 und 5 sogar 32-Bit-Zähler) dessen Zähltakt über sog. Vorteiler (Prescaler) aus dem Systemtakt abgeleitet wird.

Ein solcher Zähler stellt eine Zeitbasis (TimeBasis) bereit. Anhand dieser Uhr können verschiedene weitere Funktionen realisiert werden. Die Output-Compare Einheit eines Timers stellt eine Art Alarmfunktion dar, die zu bestimmten Zeiten Ereignisse auslösen kann. Die Input-Capture Einheit wiederum ist vergleichbar mit einer Stoppuhr mit der der Zeitpunkt des Auftretens eines Ereignisses oder die Zeitdauer von Ereignissen bestimmt werden kann. Die Timer 1 bis 5 und Timer 8 verfügen über 4 unabhängige Kanäle (Channels), die als Output-Compare, Input-Caputre, zur PWM-Generierung oder als One-pulse Mode Betrieb verwendet werden können. Timer 9 bis 14 besitzen nur 2 solcher Channel und Timer 6 und 7 gar keinen Channel.

Die von einem Timer empfangenen oder ausgesendeten Ereignisse können unterschiedlichster Natur sein. Ein Timer kann über seine Output-Compare Einheit direkt Signale an externe Pins des Controllers routen, um z.B. eine LED im Sekundentakt blinken zu lassen. Er kann aber auch Interrupts auslösen oder ADC-Wandlungen anstoßen.

Die Input-Capture Einheit dagegen dient dazu einen Zeitstempel aufzuzeichnen, sobald ein Ereignis auftritt, auf das der Timer "horcht". Ein solches Ereignis könnte z.B. durch eine externe Lichtschranke hervorgerufen werden, die ein Wettläufer beim Erreichen der Ziellinie durchtritt.

Neben der Verwendung als Zeitgeber und Zeitnehmer kann ein Timer auch noch als Ereigniszähler eingesetzt werden. Dazu wird der Zähltakt statt von der internen Systemuhr einfach von einem anderen Ereignisgeber (Trigger) abgeleitet. Damit kann man zum Beispiel zählen, wie oft eine Taste gedrückt wurde, oder wie viele Umdrehungen ein Motor gemacht hat.

7.6.1. Konfiguration eines Timers

Bevor ein Timer verwendet werden kann, muss wie immer die Komponente durch Aktivierung ihres Taktes zunächst eingeschaltet werden. Am Beispiel von Timer-1 geschieht dies mit folgendem Befehl:

```
__HAL_RCC_TIM1_CLK_ENABLE();
```



Schauen Sie sich genau Abbildung 16 im Reference Manual an. Dort ist

> beschrieben, wie die Takte im Controller abgeleitet werden. Der Takt für alle Timer wird vom Systemtakt (SystemcoreClock) abgeleitet. In unserem Fall verwenden wir einen Systemtakt von 168 MHz. Zusätzlich ist zu beachten, an welchen APBx-Takt die Timer angeschlossen sind. Die Timer 1 und 8 bis 11 erhalten ihren Takt von APB2, welcher mit maximal 90 MHz getaktet werden kann. Die Timer 2 bis 7 und 12 bis 14 hängen an APB1, welcher mit maximal 45 MHz getaktet werden kann. Durch unseren 168 MHz Systemtakt wird APB1 auf 42 MHz und APB2 auf 84 MHz getaktet, bedingt durch die APBx Prescaler von 1, 2, 4, 8 oder 16. Die Timer werden jedoch nochmals besonders behandelt und erhalten üblicherweise den doppelten Bus-Takt an dem sie angeschlossen sind. Das heißt in unserem Fall erhalten die Timer an APB1 den halben und die Timer an APB2 den vollen Systemtakt. Es ist **WICHTIG** sich bewusst zu machen, mit welchem Takt ein Timer versorgt wird, da bei Nichtbeachtung Timing-Fehler vorprogrammiert sind!

Als nächstes kann durch ausfüllen einer Initialisierungsstruktur die Zeitbasis (TimeBase) konfiguriert werden. Folgend eine Beispielkonfiguration für den Timer-1 (TIM1):

```
TIM_HandleTypeDef tim_handle_struct;                                    ①

tim_handle_struct.Instance = TIM1;                                      ②
tim_handle_struct.Init.Prescaler = (SystemCoreClock / 10000) - 1;       ③
tim_handle_struct.Init.Period = 10000 - 1;                              ④
tim_handle_struct.Init.CounterMode = TIM_COUNTERMODE_UP;                ⑤
tim_handle_struct.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;          ⑥
tim_handle_struct.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE; ⑦
tim_handle_struct.Init.RepetitionCounter = 0;                           ⑧
```

① Struktur zur Konfiguration eines Timers.

② ***Instance***: Hierüber wird bekannt gegeben, welcher Timer konfiguriert werden soll (in unserem Fall TIM1).

③ ***Init.Prescaler***: Hier wird der Takt-Vorteiler des Timers gesetzt. Der Vorteiler teilt den hohen Systemtakt (168 MHz bzw. 84 MHz) um den Wert (Prescaler+1). Ein Prescaler von 0 leitet den vollen Systemtakt zum Timer durch. Ein Prescaler von 999 würde den Systemtakt durch 1000 (999+1) teilen und führt zu einem Zähltakt von 168 MHz/1000 = 168 kHz. Der Zähler würde also 168000 mal pro Sekunde inkrementiert. Da der Systemtakt variieren kann ist es eine gute Idee, den Timer-Takt so zu berechnen, dass er nicht vom Systemtakt abhängt. Dies kann durch die Formel Prescaler = (SystemCoreClock / Freq) - 1 geschehen. Eine Frequenz von Freq = 10000 führt z.B. dazu, dass der Timer unabhängig vom Systemtakt (SystemCoreClock) immer mit 10 kHz getaktet wird. **Vor-**

sicht: Das Register, welches den Wert für den Prescaler speichert fasst nur 16-Bit Werte, d.h. Prescaler > 65535 sind nicht möglich! Achten Sie darauf, dass das Prescaler-Ergebnis eine ganze Zahl sein muss. Bei Nachkommastellen werden diese abgeschnitten – der Timer läuft dann mit einem anderen Takt als berechnet.

④ ***Init.Period***: Hier kann eingestellt werden bis zu welchem Wert der Timer zählen soll (wieviel Zeit verstreicht), bevor er wieder zurück gesetzt wird und von vorne beginnt. Angenommen der Vorteiler wurde wie zuvor beschrieben so eingestellt, dass der Timer mit einem Takt von 10 kHz versorgt wird, so würde eine Einstellung von Period = 9999 (also 10000-1) bewirken, dass der Zähler immer von 0-9999 zählt und anschließend auf 0 zurück gesetzt wird. Eine solche Periode würde dann bei einem Zähltakt von 10 kHz genau 1 Sekunde dauern. Bei einer Timereinheit gibt es kein "Period"-Register in dem dieser Wert geschrieben wird. Der hier gesetzte Wert wird in das "Auto-Reload Register" (TIMx_ARR) geschrieben.

⑤ ***Init.CounterMode***: Hier kann eingestellt werden, ob der Zähler aufwärts (TIM_COUNTERMODE_UP), abwärts (TIM_COUNTERMODE_DOWN) oder abwechselnd aufwärts und abwärts (TIM_COUNTERMODE_CENTERALIGNEDx; x = 1-3) zählen soll.

⑥ ***Init.ClockDivision***: Ein für die speziellen Modi *Dead-time generators* und *digital filters* benutzter Teiler.



Der Name dieses Felds ist etwas irreführend. Es hat keinerlei Einfluss auf den Timer und seine Frequenz in den Standard-Modi!

⑦ ***Init.AutoReloadPreload***: Wenn deaktiviert, so wird der neue zu setzende Wert für das Auto-Reload Register **sofort** in das Register geschrieben. Wenn aktiviert, so wird der neue zu setzende Wert für das Auto-Reload Register zunächst gebuffert und erst bei einem Update-Event des Timers in das Register geschrieben.

⑧ ***Init.RepetitionCounter***: Dieser Wert gibt an, nach wie vielen "Überläufen" des Timers ein Update-Event generiert werden soll. Die Anzahl der Überläufe wird mit N+1 berechnet. Hier wird also bei 0+1 = 1 nach jedem Überlauf auch ein Update-Event generiert. Bei einem Wert von 1 (1+1 = 2) würde erst ein Update-Event generiert werden, wenn der Counter 2 mal übergelaufen ist. Überlaufen bezieht sich auf einen Timer im TIM_COUNTERMODE_UP Mode. Gleiches Verhalten hätte man bei einem Unterlauf, wenn der Timer im TIM_COUNTERMODE_DOWN betrieben wird. Ist der Timer in einem TIM_COUNTERMODE_CENTERALIGNED konfiguriert, zählt hier jeder Über- und Unterlauf.

Anschließend muss die Initialisierungsstruktur noch der Initialisierungsfunktion übergeben werden, damit Timer-1 nach unseren Einstellungen konfiguriert wird.

```
HAL_TIM_Base_Init(&tim_handle_struct);
```

Damit ist Timer-1 konfiguriert, er "läuft" aber noch nicht. Zur aktivierung des Timer-1 muss separat eine Timer-Startfunktion aufgerufen werden, mit unserer Initialisierungsstruktur als Argument.

```
HAL_TIM_Base_Start(&tim_handle_struct);
```

Um den aktuellen Stand des Zählwerks auszulesen bietet die HAL folgendes Makro:

```
uint16_t time = __HAL_TIM_GET_COUNTER(&timer_handle_struct);
```

7.6.2. Konfiguration einer Output-Compare Einheit

Mit der TimeBase alleine lässt sich noch nicht allzu viel anfangen. Um beispielsweise ohne Prozessorbelastung ein Pulsweiten-Moduliertes (PWM) Signal zu erzeugen oder andere Ereignisse anzustoßen, muss zunächst eine der 4 Output-Compare (OC) Einheiten (die Alarmgeber) konfiguriert werden.

Eine OC-Einheit ist im Prinzip ein automatischer Vergleicher, der den im Output-Compare Register eingetragenen Wert mit dem aktuellen Stand des Zählwerks vergleicht. Überschreitet der Zählerwert den Vergleichswert, wird beispielsweise ein Ereignis ausgelöst. Das kann z.B. das Setzen eines PWM-Ausgangs sein, aber auch das Auslösen eines Interrupts oder einer AD-Wandlung.

Eine OC-Einheit wird durch ausfüllen der zugehörigen Initialisierungsstruktur konfiguriert. Hier am Beispiel des OC-Channels 2 von Timer 1 gezeigt:

```
TIM_OC_InitTypeDef tim_oc_handle_struct;                    ①

tim_oc_handle_struct.OCMode = TIM_OCMODE_PWM1;              ②
tim_oc_handle_struct.Pulse = 5000;                          ③
tim_oc_handle_struct.OCIdleState = TIM_OCIDLESTATE_SET;     ④
tim_oc_handle_struct.OCPolarity = TIM_OCPOLARITY_HIGH;      ⑤
tim_oc_handle_struct.OCNIdleState = TIM_OCNIDLESTATE_RESET; ⑥
tim_oc_handle_struct.OCNPolarity = TIM_OCNPOLARITY_LOW;     ⑦
tim_oc_handle_struct.OCFastMode = TIM_OCFAST_DISABLE;       ⑧
```

① Struktur zur Konfiguration eines Output-Compare-Channels für einen Timer x.

② ***OCMODE***: Der Modus in dem der OC-Channel betrieben werden soll. Er bestimmt welche Aktion ausgeführt wird, wenn der Zähler den Vergleichswert im Compareregister erreicht bzw. über- oder unterschreitet. Zur Erzeugung eines PWM-Signals wird z.B.

TIM_OCMode_PWM1 eingestellt. Die Zyklusdauer der TimeBase (hier *Period* von TIM1) bestimmt die Periodendauer des PWM-Signals, während in *Pulse* das Tastverhältnis (Duty-Cycle) eingestellt werden kann.

③ ***Pulse***: Hier wird der Vergleichs- (Compare-) Wert eingetragen. Im Falle einer PWM-Erzeugung kann hier die Puls-Dauer bzw. das Tastverhältnis (Duty-Cycle) des PWM-Signals eingestellt werden. Der Wert von *Pulse* muss im Intervall [0..*Period*] liegen.

④ ***OCIdleState***: Legt den Status des Pins fest, wenn sich die OC-Einheit im Idle-State befindet.

⑤ ***OCPolarity***: Die Polarität des Ausgabesignals kann hier via Software-Schalter auf Active-High oder Active-Low gesetzt werden. Die Einstellung Active-Low invertiert also quasi das PWM-Signal.

⑥ ***OCNIdleState***: Wie *OCIdleState*, nur für invertierten/komplementären Ausgang.

⑦ ***OCNPolarity***: Wie *OCPolarity*, nur für invertierten/komplementären Ausgang.

⑧ ***OCFastMode***: Der Fast Mode ist nur gültig, wenn der OCMode mit PWM1 oder PWM2 gewählt ist. Zusätzlich betrifft der Fast Mode (nur) den One-Pulse Mode. Für mehr Informationen zum Fast Mode studieren Sie das Reference Manual.



Der Fast Mode kann in unserem Beispiel auch erstmal ignoriert werden. Sie können ihn wahlweise *disablen* oder ganz weg lassen.

Mittels der zugehörigen Funktion wird die OC-Einheit eingestellt.

```
HAL_TIM_OC_ConfigChannel(&tim_handle_struct, &tim_oc_handle_struct,
TIM_CHANNEL_2);
```



Der OCFastMode wird in der Funktion `HAL_TIM_OC_ConfigChannel` ignoriert. Die Funktion `HAL_TIM_PWM_ConfigChannel` beachtet den OCFastMode und macht sonst exakt das Gleiche wie die Funktion `HAL_TIM_OC_ConfigChannel`. Da wir in unserm Beispiel den OCFastMode nicht benötigen, können wir bei `HAL_TIM_OC_ConfigChannel` bleiben. Beides funktioniert.

Dabei ist zu beachten, dass hier entschieden wird, welcher Channel zu welchem Timer eingestellt werden soll. Wir verwenden hier die Struktur `tim_handle_structure` um anzugeben um welchen Timer es sich handelt (in unserem Fall Timer 1 wie im vorherigen Codebeispiel angegeben). Argument zwei der Funktion ist unsere Initalisierungsstruktur zur einstellung der OC-Einheit. Argument drei gibt letztendlich den Channel an.

Die OC-Einheit ist nun eingestellt, jedoch noch nicht aktiv. Auch hierfür gibt es die passende Funktion für:

```
HAL_TIM_OC_Start(&timer_handle_struct, TIM_CHANNEL_2);
```



Die unterschiedlichen Start-Funktionen des Timers sind Alternativen zueinander und müssen nicht mehrmals aufgerufen werden. Benutzt man einen Timer beispielsweise in Verbindung mit einer OC-Einheit, so reicht es, die Funktion `HAL_TIM_OC_Start` aufzurufen, ohne vorher noch `HAL_TIM_Base_Start` aufrufen zu müssen.

Über unsere erst verwendete Struktur geben wir erneut an um welchen Timer es sich handelt. Argument zwei gibt den Channel des Timers an, der aktiviert werden soll.



Auch hier gibt es eine äquivalente Funktion zu `HAL_TIM_OC_Start`, nämlich `HAL_TIM_PWM_Start`. Zum jetzigen Stand der CubeHAL führen beide jedoch den exakt gleichen Code aus.

7.6.3. Konfiguration eines GPIO-Pins zur PWM-Ausgabe

Bevor nun endgültig ein PWM-Signal auf einem externen Pin des Controllers ausgegeben werden kann, muss dieser Pin zunächst auf den Alternate-Function Modus konfiguriert werden. Dabei ist insbesondere zu beachten, dass die Ausgänge eines bestimmten Timer-Bausteins nicht auf beliebige Pins geroutet werden können. Vielmehr ist jede OC-Einheit eines Timers prinzipiell einem bestimmten externen GPIO-Pin zugeordnet. Glücklicherweise lassen die meisten Timer zumindest eine Alternativkonfiguration zu.

Soll ein PWM-Signal auf einem bestimmten externen GPIO-Pin ausgegeben werden empfiehlt sich die folgende Vorgehensweise:

Schlagen sie im technischen Datenblatt ihres konkreten Controllers (hier STM32F429ZIT6) in "Table 10. STM32F427xx and STM32F429xx pin and ball definitions" unter ihrem gewünschten GPIO-Pin nach, welcher OC-Channel von welchem Timer auf diesen Pin geroutet werden kann.

-	61	M9	71	H5	82	K8	V_{SS}	S			
-	62	N9	72	J5	83	L9	V_{DD}	S			
41	63	R9	73	J4	84	R9	PE10	I/O	FT		TIM1_CH2N, FMC_D7, EVENTOUT
42	64	P10	74	K4	85	P10	PE11	I/O	FT		TIM1_CH2, SPI4_NSS, FMC_D8, LCD_G3, EVENTOUT
43	65	R10	75	L4	86	R10	PE12	I/O	FT		TIM1_CH3N, SPI4_SCK, FMC_D9, LCD_B4, EVENTOUT

Aus der Tabelle (und dem Bildausschnitt) lässt sich erkennen, dass der Pin, den wir für

unseren OC-Channel 2 von Timer-1 benutzen können standardmäßig auf Pin 11 des Ports E gesetzt ist. Wir müssen also diesen Pin in seiner "Alternate function" konfigurieren. Pin 11 des Ports E wird ebenfalls von unserem **8 SEG LED Boards** benutzt, dieser steuert nämlich das "Dot"-Segment an. Dies bietet sich hervorragend dafür an, dieses Segment mit unserem bisher konfigurierten Timer-1 und OC-Channel 2 zu testen.

Wie Sie einen Pin konfigurieren wissen Sie bereits. Sie müssen hier für diesen Pin noch folgende Einstellungen übernehmen.

```
GPIO_init_struct.Mode = GPIO_MODE_AF_PP;    ①
GPIO_init_struct.Alternate = GPIO_AF1_TIM1; ②
```

① Benutzt den Pin als Alternate Function im Treiber-Modus (Push/Pull).

② spezifiziert die Alternate Function, die für diesen Pin benutzt werden soll.



Durch das Verwenden und Neukonfigurieren von Pin 11 des Ports E wird die Konfiguration, die Sie in Ihrer Bibliothek für das **8 SEG LED Board** eingestellt haben, überschrieben. Dies ist aber für dieses Kapitel nicht weiter schlimm sondern bietet eine gute Möglichkeit die hiesigen Aufgaben zu den Timern zu lösen.



Vergessen Sie nicht den Timer sowie die OC-Channel zu starten, nachdem Sie alles Notwendige (Timer, OC-Channel, PWM-Pin) konfiguriert haben.

7.6.4. Einstellung von Tastverhältnis und Periodendauer

Typischerweise möchte man nach Konfiguration eines PWM-Timers aus dem Programm heraus dessen Tastverhältnis (DutyCycle) einstellen, um z.B. eine LED zu dimmen oder die Drehzahl eines Motors zu verändern.

Dies geschieht durch Setzen eines neuen Wertes im Compare-Register der zugehörigen Output-Compare-Einheit des Timers. Folgendes Makro hilft dabei:

```
__HAL_TIM_SET_COMPARE(&timer_handle_struct, TIM_CHANNEL_2, 3000);
```

Darüber hinaus kann auch die Periodendauer (Frequenz) des PWM-Signals verändert werden. Dies kann durch eine Änderung des Timer-Taktes geschehen. Dazu kann dessen Takt-Vorteiler (Prescaler) on the fly verändert werden. Auch hierfür ein Makro:

```
__HAL_TIM_SET_PRESCALER(&timer_handle_struct, (SystemCoreClock/15000)-1);
```

Dabei ist zu beachten, dass der neue Wert für den Prescaler erst nach einem Update-Event (Überlauf) des Timers gesetzt wird.



Bearbeiten Sie nun am besten die Aufgaben im Kapitel **Timer** im dritten Aufgabenblatt `abgabe3.pdf`

7.7. Nested Vector Interrupt Controllers (NVIC)

Lernziele:

- Grundlegendes Verständnis des Nested Vector Interrupt Controllers (NVIC)
- Priorisierung von Interrupts
- Konfiguration von Interruptquellen
- Verständnis der Interrupt-Vektor-Tabelle
- Verständnis des Interrupt-Handlings in Verbindung mit der CubeHAL
- Erstellung von Interrupt-Service-Routinen (ISR)

Der Cortex-M4 verfügt über einen sehr effizienten Interrupt-Controller. Dieser ist beim M4 sehr eng mit dem Kern verbunden. Die Sicherung des Kontextes wird vom Prozessor automatisch vorgenommen, weshalb der Einsprung in eine Interrupt-Service-Routine gerade einmal 12 Prozessortakte benötigt.

Allgemein kann jedem Interrupt eine Priorität zugewiesen werden, d.h. die Priorität wird nicht über die Position in der Vektortabelle fest vorgegeben. Die Prioritäten können in Gruppen organisiert werden. Interrupts in verschiedenen Gruppen können sich gegenseitig unterbrechen. Innerhalb einer Gruppe unterbricht ein Interrupt höherer Priorität jedoch keinen niederpriorigen. Hier dient die Priorität der Reihenfolge der Abarbeitung, falls mehrere Interrupts getriggert wurden. Das ist z.B. wichtig, um Deadlocks bei gemeinsam verwendeter Hardware zu vermeiden.

7.7.1. Funktionsweise

Alle Interrupts, einschließlich der Prozessorkern-Interrupts, werden von dem NVIC verwaltet. In ihm wird mittels Ein- und Ausschalten der Interrupt Kanäle festgelegt, welche Peripherie Interrupts erzeugen darf. Die Vektortabelle enthält 91 Einträge (+16 wenn der Controller eine FPU hat), wobei jede Interrupt-Quelle einen festen Vektor mit einem Eintrag der entsprechenden Interrupt Service Routine (ISR) hat. Wenn man die Interrupt Vektor Tabelle also als Array mit Adressen von Interrupt Service Routinen betrachtet, hat jede Peripherie einen Index, an dem die Adresse der jeweiligen ISR abgelegt werden kann. Die Interrupts können in Gruppen mit unterschiedlichen Prioritäten verschachtelt (engl. nested) werden. Innerhalb dieser Gruppe ist es möglich den Interrupts weitere Prioritäten

zuzuweisen. Sollten die einzelnen Interrupts innerhalb ihrer Gruppe die selbe Priorität haben, so wird ihre Priorität durch ihre Position in der Vektor Tabelle festgelegt. Aus jener lässt sich auch die Kanal-Nummer (Position) entnehmen. Für Priorität und Subpriorität stehen zusammen 4 Bit zur Verfügung. Je höher der jeweilige Wert, desto niedriger seine Priorität. Eine Ausnahme bei der Priorisierung bilden drei Core Exceptions (Reset, None Maskable Interrupt (NMI), Hard Fault). Diese haben in der angegebenen Reihenfolge die höchsten Prioritäten, welche auch nicht verändert werden können. [12], [8]

7.7.2. Pre-Emption Priority

Bevor der NVIC konfiguriert werden kann, muss einmal die gewünschte Pre-Emption-Priority-Group eingestellt werden. Das Wort Pre-Emption bedeutet soviel wie "Vorwegnahme". Gemeint ist hier die Aufteilung der 16 (4-Bit) möglichen Interruptprioritäten in eine Anzahl von Prioritätsgruppen. Innerhalb dieser Gruppen können wieder Unterprioritäten vergeben werden.

Die Einstellung geschieht mittels

```
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_x);
```

Möglich sind die Gruppen 0-4, wobei diese Zahl letztlich für die Anzahl der Bits steht, die für die Pre-Emption-Priority verwendet werden.

NVIC_PRIORITYGROUP_0 0 Bits Pre-Emption, 4 Bits Sub-Priority

NVIC_PRIORITYGROUP_1 1 Bits Pre-Emption, 3 Bits Sub-Priority

NVIC_PRIORITYGROUP_2 2 Bits Pre-Emption, 2 Bits Sub-Priority

NVIC_PRIORITYGROUP_3 3 Bits Pre-Emption, 1 Bits Sub-Priority

NVIC_PRIORITYGROUP_4 4 Bits Pre-Emption, 0 Bits Sub-Priority

Auf diese Weise wird es möglich, Interrupts in Gruppen bestimmter Priorität einzuordnen. Innerhalb ihrer Gruppe erhält sie eine Sub-Priorität. Tritt während der Ausführung einer ISR ein Interrupt höherer Priorität (nicht Sub-Priorität) auf, so wird die Ausführung unterbrochen und die ISR des höher priorisierten Interrupt angesprungen. Interrupts niedrigerer Priorität können höher priorisierte Interrupts nicht unterbrechen, ihre Ausführung wird dann zurück gestellt.

Beispiel 1. Prioritäten

Wir möchten gerne unsere ISR in zwei Gruppen einteilen. Einmal hoch priorisierte

kurze ISRs für Echtzeitaufgaben (ABS Bremssystem, Airbag) und andererseits gering priorisierte ISRs (Fensterheber, Blinker). Hier bietet sich die Aufteilung in zwei Vorzugs-Gruppen (Einstellung: *NVIC_PRIORITYGROUP_1*) an. Innerhalb der höchst priorisierten Gruppe 0 (Echtzeit) könnte z.B. der Airbag eine höhere Subpriorität (kleinere Nummer), wie das ABS haben. In Gruppe 1 könnte der Blinker eine höhere Subpriorität als der Fensterheber haben. Alle Interrupts der Gruppe 0 unterbrechen dabei die Interrupts der Gruppe 1. Bei den Subprioritäten gibt es dabei keine Verdrängung, die Subprioritäten bestimmen nur die Reihenfolge falls mehrere Interrupts auf eine Behandlung warten. Das heißt: Der Blinker kann den Fensterheber nicht verdrängen. Falls jedoch gerade die ABS Interrupt Service Routine ausgeführt wird und währenddessen der Blinker und der Fensterheber einen Interrupt werfen, würde der Blinker Interrupt anschließend als erstes behandelt werden.

7.7.3. Konfiguration des NVIC

Das Konfigurieren eines Interrupts findet durch folgenden Aufruf statt:

```
// void HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority,
uint32_t SubPriority);

HAL_NVIC_SetPriority(TIM2_IRQn, 1, 6);  ①
HAL_NVIC_EnableIRQ(TIM2_IRQn);          ②
```

① Konfigureren des TIM2 Interrupts auf die Prioritätsgruppe 1, mit der Subpriorität 6

② Aktivieren des Interrupts für TIM2

7.7.4. Konfigurieren einer Interruptquelle

Den NVIC zu konfigurieren reicht noch nicht aus, um Interrupts auslösen zu können. Interrupts können von fast allen Peripheriekomponenten ausgelöst werden, um bestimmte Zustände zu signalisieren oder Aktionen auszulösen.

Damit eine Komponente Interrupts auslösen kann, muss jedoch zuerst die zugehörige Interruptquelle der Peripheriekomponente aktiviert werden.

Um z.B. den Update-Interrupt von Timer *TIM2* zu aktivieren bietet STM32Cube bereits folgendes Makro:

```
__HAL_TIM_ENABLE_IT(htim, TIM_IT_UPDATE)
```

Der erste Parameter beschreibt einen Zeiger auf das Timer-Peripherie-Handle. Der zweite

Parameter spezifiziert die konkrete Interruptquelle, die bei dieser Peripherie aktiviert werden soll. Im Beispiel wird das *Timer-Update* als Interruptquelle aktiviert, so dass bei jedem Überlauf des Timers ein Interrupt ausgelöst wird.

Um diese Interruptquelle wieder zu deaktivieren, wäre folgendes Makro zu verwenden:

```
__HAL_TIM_DISABLE_IT(htim, TIM_IT_UPDATE)
```

Die Makros anderer Peripheriekomponenten sind nach dem gleichen Schema benannt: z.B. *HAL_DMA_ENABLE_IT(...),* HAL_ADC_ENABLE_IT(...) oder __HAL_USART_ENABLE_IT(...).

Wenn Sie eine Interruptquelle einer Peripherie nutzen möchten, lohnt es sich die ****_Start_IT(...)* Funktion der Peripherie zu betrachten. Dies ist immer eine Variante, die Peripherie mit Interrupt zu starten. Wichtig ist dabei jedoch, dass darin der Interrupt aktiviert wird, den Sie auch verwenden wollen. Bei näherer Betrachtung der Funktion *HAL_TIM_Base_Start_IT(...)* fällt beispielsweise auf, dass der Update-Interrupt bereits aktiviert wird. Somit würden Sie sich das Makro sparen können:

```
 /**
  * @brief  Starts the TIM Base generation in interrupt mode.
  * @param  htim TIM Base handle
  * @retval HAL status
  */
HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim)
{
  uint32_t tmpsmcr;

  /* Check the parameters */
  assert_param(IS_TIM_INSTANCE(htim->Instance));

  /* Enable the TIM Update interrupt */
  __HAL_TIM_ENABLE_IT(htim, TIM_IT_UPDATE);         ①

  /* Enable the Peripheral, except in trigger mode where enable is
     automatically done with trigger */
  tmpsmcr = htim->Instance->SMCR & TIM_SMCR_SMS;
  if (!IS_TIM_SLAVEMODE_TRIGGER_ENABLED(tmpsmcr))
  {
    __HAL_TIM_ENABLE(htim);                         ②
  }

  /* Return function status */
```

```
  return HAL_OK;
}
```

① Aktiviert den Timer Update Interrupt

② Starte die Timer Peripherie

Eine Liste der unterschiedlichen Interruptquellen, die eine bestimmte Peripheriekomponente anbietet, entnimmt man am besten der jeweiligen Header Datei. Bei der Makrodefinition sind die möglichen Argumente mit aufgezählt. So steht in der *stm32fxx_hal_tim.h* beispielsweise:

```
/** @brief  Enable the specified TIM interrupt.
  * @param  __HANDLE__ specifies the TIM Handle.
  * @param  __INTERRUPT__ specifies the TIM interrupt source to enable.
  *          This parameter can be one of the following values:
  *            @arg TIM_IT_UPDATE: Update interrupt
  *            @arg TIM_IT_CC1:   Capture/Compare 1 interrupt
  *            @arg TIM_IT_CC2:  Capture/Compare 2 interrupt
  *            @arg TIM_IT_CC3:  Capture/Compare 3 interrupt
  *            @arg TIM_IT_CC4:  Capture/Compare 4 interrupt
  *            @arg TIM_IT_COM:   Commutation interrupt
  *            @arg TIM_IT_TRIGGER: Trigger interrupt
  *            @arg TIM_IT_BREAK: Break interrupt
  * @retval None
  */
#define __HAL_TIM_ENABLE_IT(__HANDLE__, __INTERRUPT__)    ((__HANDLE__)-
>Instance->DIER |= (__INTERRUPT__))
```

7.7.5. Interrupt Service Routine

Nachdem der NVIC konfiguriert und die gewünschte Interruptquelle aktiviert ist, muss jetzt eine Interrupt Service Routine (ISR) erstellt werden, die auftretende Interrupt-Requests (IRQ) bearbeitet.

Aus Gründen der sauberen Programmstruktur werden alle ISRs in einem eigens dafür vorgesehenen Programmmodul abgelegt. Dieses Modul finden Sie in ihrem Projekt im src-Verzeichnis: *stm32f4xx_it.c*

In dieser Datei finden Sie bereits die Behandlung für den SysTick Interrupt.

```
void SysTick_Handler(void)
{
```

```
    HAL_IncTick();              ①
    HAL_SYSTICK_IRQHandler();   ②
}
```

① Inkrementieren des Systemlaufzeit-Zählers von der STM32CubeHAL

② Behandlung des Interrupts durch die STM32CubeHAL

Den Namen *SysTick_Handler* findet man ebenfalls in der Datei *startup_stm32f429xx.s*. In dieser Datei wird die Interrupt Vektor Tabelle mit Inhalt befüllt. Diese Tabelle wird beim Auftreten eines Interrupts vom Prozessor herangezogen, um die Behandlungs-Routine (also die ISR) für den IRQ auszuwählen.

```
g_pfnVectors:
    .word  _estack
    .word  Reset_Handler

    .word  NMI_Handler
    .word  HardFault_Handler
    .word  MemManage_Handler
    .word  BusFault_Handler
    .word  UsageFault_Handler
    .word  0
    .word  0
    .word  0
    .word  0
    .word  SVC_Handler
    .word  DebugMon_Handler
    .word  0
    .word  PendSV_Handler
    .word  SysTick_Handler            ①

    [...]

    .word     TIM1_BRK_TIM9_IRQHandler
    .word     TIM1_UP_TIM10_IRQHandler
    .word     TIM1_TRG_COM_TIM11_IRQHandler
    .word     TIM1_CC_IRQHandler
    .word     TIM2_IRQHandler                  ②

    [...]
```

① Eintrag der SysTick ISR

② Eintrag der TIM2 ISR

In der Interrupt Vektor Tabelle (g_pfnVectors) sind Funktionsnamen hinterlegt. Wie am Beispiel von TIM2 zu sehen ist, können auch mehrere IRQs durch eine ISR behandelt werden. Nicht jeder Interrupt wird zwangsweise auf eine eigene ISR geleitet. Ansonsten würden die anderen Interrupts von TIM2 ebenfalls aufgezählt werden (UPDATE, CC1-4, COM, TRIGGER, BREAK). Diese Bündelung der IRQs ist auch bei Timer 1 zu sehen, der sich z.B. die ISR für den BREAK IRQ mit den IRQs von TIM9 teilt. Die Funktionsnamen, die in dieser Tabelle genutzt werden, werden weiter unten in der Datei als *weak* deklariert und sozusagen auf die Implementierung der Funktion *Default_Handler* umgelenkt.

```
/*******************************************************************************
**
*
* Provide weak aliases for each Exception handler to the Default_Handler.
* As they are weak aliases, any function with the same name will override
* this definition.
*
*******************************************************************************
*/

    [...]

    .weak      SysTick_Handler
    .thumb_set SysTick_Handler,Default_Handler

    [...]

    .weak      TIM2_IRQHandler
    .thumb_set TIM2_IRQHandler,Default_Handler
```

Dieses *weak* bezieht sich auf den Linker, der für das binden der Funktionsaufrufe zu den Funktionen übernimmt. Die Aussage, die hier getroffen wird ist: Gibt es keine Funktionsdefinition für *TIM2_IRQHandler*, dann soll die Funktion *Default_Handler* als Definition dienen. Damit verweisen alle Einträge auf den Default Handler, bis eine Funktion mit diesem Namen definiert wird, wie es beispielsweise für den *SysTick_Handler* bereits in der Datei *stm32f4xx_it.c* getan wurde.

Der Default Handler sieht dabei folgendermaßen aus:

```
/**
 * @brief  This is the code that gets called when the processor receives an
 *         unexpected interrupt.  This simply enters an infinite loop,
preserving
 *         the system state for examination by a debugger.
```

```
 * @param  None
 * @retval None
*/
    .section  .text.Default_Handler,"ax",%progbits
Default_Handler:
Infinite_Loop:
  b  Infinite_Loop
  .size  Default_Handler, .-Default_Handler
```

Der Default Handler ist demnach nur eine Endlosschleife, die ausgeführt wird, wenn ein Interrupt auftritt, dessen ISR nicht explizit vom Programmierer definiert also implementiert wurde.



Wenn Sie Interrupts nutzen wollen, lohnt es sich einen Breakpoint im Default Handler zu setzen, um zu sehen, ob Ihr Programm auch alle auftretenden Interrupts behandelt. Falls Sie alles richtig machen, wird Ihr Programm nicht in diesen Breakpoint laufen.

Um eine eigene ISR zu implementieren, müssen Sie also die Funktion aus der Interrupt Vektor Tabelle implementieren. Nehmen wir das bisherige Beispiel für den Update-IRQ von Timer 2, so muss die Funktion *TIM2_IRQHandler* implementiert werden. Diese sollten Sie in der Datei *stm32f4xx_it.c* implementieren. Eine ISR hat weder Parameter noch Rückgabewert.

```
void TIM2_IRQHandler(void)
{
    /* TIM Update event */
    if (__HAL_TIM_GET_FLAG(&htim2, TIM_FLAG_UPDATE) != RESET)     ①
    {
        __HAL_TIM_CLEAR_IT(&htim2, TIM_IT_UPDATE);               ②

        [...]
    }
}
```

① Prüfung, ob das Update Interrupt Pending Flag bei Timer 2 gesetzt ist

② Löschen des Update Interrupt Pending Flags



Ganz wichtig ist, dass innerhalb der Routine durch Löschen des entsprechenden Interrupt Pending Bits dem Controller mitgeteilt wird, dass der Interrupt abgearbeitet wurde. Tut man dies nicht, so wird der gleiche Interrupt nach Rücksprung aus der ISR sofort wieder ausgelöst und bear-

> beitet. Dies würde zu einer endlosen Abarbeitung des Interrupts und damit zur Blockade des Programms führen. Dieses löschen des Bits können Sie zu Beginn oder Ende Ihrer ISR durchführen, je nachdem, ob Sie während der Bearbeitung einen weiteren Interrupt erkennen wollen oder erst wieder im Anschluss.

Wie bereits beschrieben, ist es häufig so, dass sich mehrere Interruptquellen einer oder mehrerer Peripherien den gleichen IRQ-Channel (also den Tabelleneintrag in dem Interrupt Vector) und damit auch die gleiche ISR teilen. Aus diesem Grund wird meist innerhalb der ISR eine entsprechende Fallunterscheidung vorgenommen. Im obigen Beispiel wird geprüft, ob tatsächlich die Interrupt Quelle *TIM_IT_UPDATE* ausgelöst hat, um dann den zugehörigen Code auszuführen. Eine andere Quelle, die in diesem Fall die ISR ausgelöst haben könnte wäre z.B. einer der vier Capture-Compare-Einheiten des Timers TIM2, die mit eigenem Code behandelt werden müssten.

Durch die STM32CubeHAL wird Ihnen einiges an Arbeit und Kontrolle abgenommen. Damit die HAL richtig funktioniert, müssen Sie die Interrupts, die Sie verwenden in die HAL weiterleiten. Für die Behandlung von Timer Interrupts gibt es z.B. die Funktion *HAL_TIM_IRQHandler*, die Sie aus Ihrer ISR aufrufen sollten.

```
void TIM2_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&htim2); ①
}
```

① Aufruf des STM32CubeHAL Timer IRQ Handlers mit dem Timer TIM2 Handle als Parameter, damit die Funktion weiß, welche Peripherie einen Interrupt ausgelöst hat.

Die Namensgebung ist dabei wieder über die Peripherien gleich *HAL_<peripherie>_IRQHandler(...)*. In dem Timer IRQ Handler der STM32CubeHAL werden bereits die Flags abgefragt und gelöscht. Der Abschnitt für den Update Interrupt sieht z.B. folgendermaßen aus:

```
  /* TIM Update event */
  if (__HAL_TIM_GET_FLAG(htim, TIM_FLAG_UPDATE) != RESET)
  {
    if (__HAL_TIM_GET_IT_SOURCE(htim, TIM_IT_UPDATE) != RESET)
    {
      __HAL_TIM_CLEAR_IT(htim, TIM_IT_UPDATE);
#if (USE_HAL_TIM_REGISTER_CALLBACKS == 1)
      htim->PeriodElapsedCallback(htim);
#else
      HAL_TIM_PeriodElapsedCallback(htim);          ①
```

```
#endif /* USE_HAL_TIM_REGISTER_CALLBACKS */
    }
  }
```

① Aufruf der HAL_TIM_PeriodElapsedCallback Funktion. Diese ist ebenfalls als Weak beschrieben und wird somit von Ihnen als Nutzer der Stm32CubeHAL implementiert.

Wie zu sehen ist, wird in diesem IRQHandler der STM32CubeHAL bereits wie empfohlen mit den Interrupt Pending Bits umgegangen. Was außerdem zu sehen ist, ist ein Aufruf einer Callback Funktion. Dieser Aufruf ist dafür da, dass wieder Nutzer-Code aufgerufen werden kann. Wenn Sie die STM32CubeHAL nutzen und den Timer Interrupt durch die HAL durchleiten, sodass Sie sich nicht um die Interrupt Auswertung kümmern müssen, müssen Sie diese Callback Funktion implementieren. Darin behandeln Sie den Interrupt im Sinne Ihres Programms. Nehmen wir wieder das Beispiel vom Timer TIM2, so könnte die Funktion folgendermaßen aussehen:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim == &htim2)          ①
    {
        // do some stuff
    }
}
```

① Sie müssen prüfen, von welcher Peripherie der Interrupt stammt. Dies wurde in der ISR beim Aufruf der HAL_TIM_IRQHandler Funktion angegeben. Wenn TIM1 auch genutzt wird, würde in der ISR ebenfalls HAL_TIM_IRQHandler aufgerufen werden. Damit in der Callback-Funktion der Ursprung ermittelt werden kann, wird dies als Parameter der Funktion übergeben.



Sie erfahren u.a. aus der Beschreibung im Kopf der STM32CubeHAL C-Datei der Peripherie eine Beschreibung der Callback Funktionen.



Schauen Sie sich die Implementierung der IRQ Handler in der STM32CubeHAL immer an, bevor Sie diese nutzen. Es gibt Ausnahmen, wo z.B. die Interrupt Pending Bits nicht gelöscht werden.

Die Funktionen und Symbole, welche Ihnen zur Verfügung stehen, finden Sie zum NVIC ausnahmsweise in der **stm32f4xx_hal_cortex.c/h**. Dies hat damit zu tun, dass es sich hier nicht um eine ST-Peripherie handelt, sondern der NVIC direct mit dem Kern verbündelt ist.

7.7.6. External Interrupt Line Controller (EXTI)

Im Zusammenhang mit dem NVIC existiert eine weitere interessante Peripherie-Einheit, der External-Interrupt-Line Controller (EXTI)

Die Einheit enthält 23 EXTI-Lines, mit so genannten Flankendetektoren, um Interrupt-Requests oder Events zu generieren, sobald sich beobachtete externe Signale ändern. Auf diese Weise kann z.B. ein Taster überwacht werden, um bei dessen Betätigung einen Interrupt auszulösen.

Um durch ein Signal, wie z.B. eine steigende Flanke auf einem externen Pin einen Interrupt auszulösen, muss zunächst der zugehörige GPIO-Pin auf den Mode *GPIO_MODE_IT_RISING* konfiguriert werden.

In der *HAL_GPIO_Init()* Funktion wird der EXTI dann mit den zu konfigurierenden Pins verbunden.

```
GPIO_InitTypeDef gpio_init = {
  .Pin = GPIO_PIN_7,
  .Mode = GPIO_MODE_IT_RISING,
  .Pull = GPIO_NOPULL,
};

HAL_GPIO_Init(GPIOB, &gpio_init);
```

Hier wird der GPIO in Verbindung mit dem EXTI so konfiguriert, dass der EXTI auf eine steigende Flanke am Pin 7 durch Auslösen eines Interrupts reagiert. Bei einem Taster muss darauf geachtet werden, ob der Taster das Signal auf Masse oder auf 3.3V zieht, also wo das andere Ende des Schalters angeschlossen ist. Entsprechend der Verschaltung muss für einen Tastendruck auf die fallende oder steigende Flanke reagiert werden.

Die EXTI-Lines sind von Ihrer Nummerierung fest mit GPIO-Pins verbunden. GPIOx Pin 0 ist auf die EXTI-Line 0 geführt und GPIOx Pin 15 ist auf die EXTI-Line 15 geführt. Dies ist im Reference Manual in Abbildung 43 zu sehen. Lesen Sie sich die Beschreibung des EXTI im Reference Manual durch.

Damit ein Interrupt Request beim Betätigen eines Tasters bearbeitet wird, muss noch der zugehörige IRQ-Channel des NVIC konfiguriert und aktiviert werden. Dazu ist zunächst der Name des IRQ-Channels zu ermitteln. Hierzu konsultiert man wieder die Beschreibung der Vektor-Tabelle im Reference Manual. dort findet man die Channels EXTI0 - EXTI4. Nach EXTI7 sucht man jedoch vergeblich. Dies kommt daher, dass die Interrupt-Quelen EXTI5-EXTI9 zu einem Interrupt Channel zusammengefasst wurden. Zu finden sind sie dann unter dem Acronym EXTI9_5.

Daraus leitet sich dann der Name des IRQ-Channels mit *EXTI9_5_IRQn* ab. Wenn man in die Interrupt Vektor Tabelle in der Datei *startup_stm32f429xx.s* an dieser Stelle schaut, ist dort die Funktion *EXTI9_5_IRQHandler* als ISR eingetragen. Dieser Interrupt muss scharf geschaltet werden. Darum kümmert sich die STM32Cube-HAL an dieser Stelle nicht:

```
HAL_NVIC_SetPriority(EXTI9_5_IRQn, 1, 0);
HAL_NVIC_EnableIRQ(EXTI9_5_IRQn);
```

Nun wird ein Interrupt bei Tastetdruck ausgelöst. Dieser Interrupt muss allerdings durch eine zu implementierende ISR behandelt werden, da ansonsten der Default Handler ausgeführt wird. Der Interrupt muss in die STM32Cube-HAL weitergeleitet werden, damit diese sich um die Bearbeitung kümmert und die Register des Controllers entsprechend verwaltet.

```
void EXTI9_5_IRQHandler(void)
{
  EXTI_HandleTypeDef hexti;
  HAL_EXTI_GetHandle(&hexti, EXTI_LINE_7);              ①
  if(HAL_EXTI_GetPending(&hexti, EXTI_TRIGGER_RISING))  ②
  {
    HAL_GPIO_EXTI_IRQHandler(GPIO_Pin_7);               ③
  }
}
```

① hexti wird mit den notwendigen Informationen gefüllt, um ein Handle für die EXTI Line 7 zu erzeugen.

② Das Pending Bit im EXTI Controller wird abgefragt, ob ein Ereignis des EXTI für EXTI Line 7 mit steigender Flanke aufgetreten ist.

③ Interruptbehandlung an die STM32Cube-HAL weiterleiten, dort wird sich um die Handhabung des NVIC gekümmert.

Möchte man selbst den Interrupt behandeln, so muss die Funktion *HAL_GPIO_EXTI_Callback* implementiert werden. Diese wird vom *HAL_GPIO_EXTI_IRQHandler* aufgerufen.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  switch(GPIO_Pin)
  {
    case GPIO_Pin_7:
      ...
      break;
```

```
    default:
      break;
  }
}
```

7.7.7. Richtlinien zu ISRs

Der NVIC des Cortex-M4 nimmt Ihnen die meisten typischen Probleme bei der Interruptbehandlung ab. Beispielsweise müssen Sie beim Eintritt in eine ISR weder den Prozessorzustand sichern, noch die Ausführung weiterer Interrupts deaktivieren. Einige Richtlinien sollten Sie aber dennoch beachten. Dies gilt selbstverständlich auch für die entsprechenden Callbacks, die aus der STM32Cube-HAL aufgerufen werden, da diese aus dem Interrupt Kontext aufgerufen werden.

1. ISRs sollten kurz sein: Eine ISR sollte nur zeitkritische Dinge behandeln und so kurz wie möglich sein. Beispielsweise gehört eine Ausgabe auf ein Display (da dies sehr zeitaufwändig ist) niemals in eine ISR. Stattdessen kann durch Setzen eines Flags eine Nachricht an die Hauptschleife des Programms gesendet werden, wo diese langsame Aktion geringer Priorität dann ausgeführt werden kann.
2. Bedenken Sie, dass ISRs den Programmablauf an beliebiger Stelle unterbrechen und auch selbst wieder unterbrochen werden können. Das heißt, bei Zugriff auf globale Daten aus einer ISR heraus riskieren Sie, dass diese ihre Konsistenz verlieren, insbesondere wenn der Zugriff nicht atomar ist. Treffen Sie also geeignete Vorsichtsmaßnahmen (Synchronisationsmechanismen, etc.)
3. Wenn Sie auf globale Variablen aus einer ISR zugreifen, denken Sie daran volatile in der Deklaration der Variable zu verwenden, damit diese nicht weg optimiert wird. Der Compiler kennt das Laufzeitverhalten durch Interrupts nicht und je nachdem, ob die ISR als Quelle oder Senke der Daten genutzt wird, kann es sein, dass der Compiler vermutet, dass nie sinnvoll auf die Daten zugegriffen wird.



Bevor Sie ab hier in der Praktikumsanleitung fortfahren, sollten Sie die Aufgaben im Kapitel **NVIC** im dritten Aufgabenblatt `abgabe3.pdf` bearbeiten und somit die dritte Abgabe abschließen.

Kapitel 8. Projekt 1: Lüfterregelung

Lernziele:

Anwendung des erlernten Grundlagenwissens in einem kleinen Projekt. Wenden sie ihr Wissen aus den letzten Kapiteln an und entwickeln sie eine Drehzahlregelung für einen PC-Lüfter.

8.1. Ansteuerung von PC-Lüftern

Die Drehzahl von Motoren ist bei konstant zugeführter Leistung lastabhängig. Das bedeutet, dass die Drehzahl von der Belastung des Motors abhängt und diese kann Aufgrund äußerer Störungen (Reibung, Druckunterschiede, etc.) variieren. Um eine exakte Regelung der Drehzahl zu erreichen, muss diese gemessen und die zugeführte Motorleistung durch einen Regelalgorithmus nachgestellt werden. Dabei muss die Leistung bei zu geringer Drehzahl etwas erhöht und bei zu hoher Drehzahl zurück genommen werden. Genauere Informationen zu den Grundlagen der Regelungstechnik erfahren Sie in der entsprechenden Präsentation und dem dazugehörigen Foliensatz.

8.1.1. Tachometerausgang

Ein Tachometer in Form eines Impulsgebers ist bereits im Lüfter integriert. Der Impulsgeber erzeugt jede halbe Umdrehung des Lüfterrades einen digitalen Rechteckimpuls - pro ganze Umdrehung also 2 Impulse.

Der Zeitraum t zwischen 2 Impulsen entspricht also der Zeit, die das Lüfterrad gerade für eine halbe Umdrehung benötigt. Für eine ganze Umdrehung benötigt das Rad die doppelte Zeit, also 2t.

Der Kehrwert der Zeit 2t, die für eine Radumdrehung benötigt wird, entspricht damit genau der Drehfrequenz in Umdrehungen pro Sekunde:

$$f_{rot} = \frac{1}{2t}$$

Üblicher ist die Angabe von Umdrehungen pro Minute (rpm):

$$f_{rot} = \frac{60}{2t}$$

8.1.2. Eingang zur Einstellung der Motorleistung

Auch ein elektronisches Stellglied zum Einstellen der Motorleistung ist bereits im Lüfter enthalten. Das Steuersignal für die Motorleistung ist ein PWM-Signal einer Frequenz im

Bereich 21-28 kHz (nominal 25 kHz). Das Tastverhältnis des PWM-Signals stellt die Leistung ein. 0% Einschaltzeit entspricht dabei der minimalen Motorleistung und 100 % Einschaltzeit des Signals der maximalen Motorleistung.



So zumindest die Theorie. In Wirklichkeit können viele Lüfter mit einer Einschaltzeit von unter 20-25% nichts weiter Anfangen und schalten entweder ab oder drehen voll auf.

8.1.3. Anschluss des Lüfters

Der Stecker des Lüfters enthält vier Pins mit folgender Pinbelegung:

Tabelle 2. Pinbelegung des Lüfters

Pin	Belegung	Farbe der Leitung am Lüfterstecker
1	GND	schwarz
2	+5V	gelb
3	Tacho-Ausgang	grün
4	PWM-Eingang	blau

Pin 1 des Lüftersteckers ist mit einer **1** markiert.

8.1.4. Konfiguration der GPIO-Pins zur Lüftersteuerung

Bei der Ansteuerung des Lüfters ist zu beachten, dass dieser über so genannte Open-Drain Anschlüsse verfügt. Der Vorteil hierbei ist, dass die verbundenen Geräte mit unterschiedlichen Versorgungsspannungen betrieben werden können. So arbeitet hier der Lüfter beispielsweise mit Logik-Pegeln von 5V, während der Cortex-M4 nur mit Pegeln von 3.3V arbeitet.

Für sie als Programmierer bedeutet das nur, dass sie Ein- und Ausgangspins des Cortex-M4 entsprechend konfigurieren müssen.

Der Tacho-Ausgang des Lüfters wird an einen GPIO-Input-Pin mit Pull-Up-Widerstand angeschlossen. Der Cortex zieht hierbei seinen Eingangspin über einen internen Pull-Up-Widerstand auf einen logischen Hi-Pegel von 3.3V. Der Lüfter zieht diesen Pegel während eines Tachoimpulses auf 0V (Lo-Pegel) herunter.

Der PWM-Eingang des Lüfters (zur Einstellung der Leistung) ist an einen Pin angeschlossen, der mittels eines Timers ein PWM-Signal ausgibt. Der Pin-Mode **muss** hier auf `GPIO_MODE_AF_OD` eingestellt werden. OD steht für Open-Drain und bedeutet, dass der Cortex den

Pin nur auf Logisch-Lo ziehen kann, nicht jedoch auf Hi. Die Herstellung des Hi-Pegels wird diesmal von einem Pull-Up Widerstand im Lüfter besorgt und der Hi-Pegel entspricht demnach 5V statt 3.3V.



Vergewissern sie sich, dass die Pins die sie verwenden möchten auch wirklich "frei" sind, also nicht schon mit anderer Elektronik auf dem Discovery-Board verbunden ist. Informationen über die Pinbelegung auf dem Board finden sie im Datenblatt **discovery-kit_user-manual.pdf** (kleiner Tip: Tabelle 6).



Bevor Sie ab hier in der Praktikumsanleitung fortfahren, sollten Sie die Aufgaben des vierten Aufgabenblatts `abgabe4.pdf` bearbeiten und somit die vierte Abgabe abschließen.

Kapitel 9. Projekt 2

Für das folgende Projekt ist mehr Eigenarbeit und Recherche nötig. Es gilt zwei Komponenten in Betrieb zu nehmen, um eine Art Knoten einer Wetterstation umzusetzen, der seine Wetterdaten mit den anderen Knoten (die Boards der anderen Gruppen) austauscht:

1. Den Umgebungssensor BME280 von Bosch, um Temperatur, Luftfeuchtigkeit und Luftdruck in Erfahrung zu bringen.
2. Die CAN-Peripherie des Cortex-M4, welche verwendet werden soll, um die Wetterdaten über den von uns zur Verfügung gestellten CAN-Bus zu veröffentlichen.

9.1. Anbindung des BME280-Umgebungssensors über den I^2C-Bus

Lernziele:

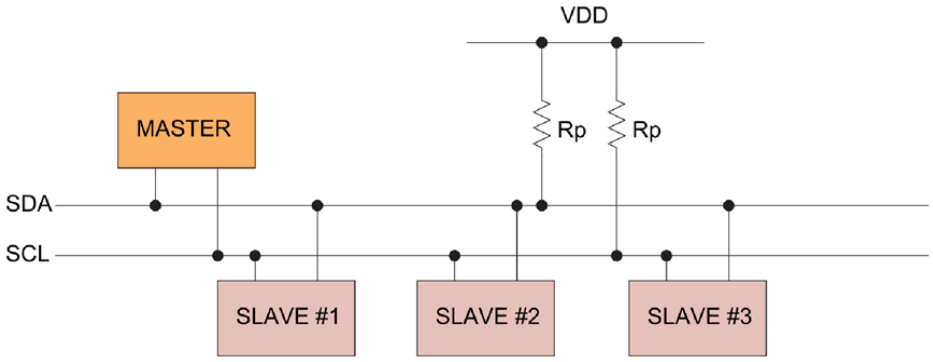
- Grundlegendes Verständnis des I^2C-Busses
- Konfiguration des Cortex-M4 I^2C-Interfaces
- Anbindung des BME280 über die bereitgestellte Bosch-Bibliothek

I^2C (Inter-Integrated Circuit) ist ein synchroner Zweidraht-Bus (SDA: Datenleitung, SCL: Clock-Leitung) und ein verbreiteter Industriestandard für Steuerungs-, Diagnose- und Überwachungsaufgaben in eingebetteten Systemen. An den in Master-/Slavearchitektur aufgebauten Bus können bis zu 112 Teilnehmer angeschlossen werden, die mit Geschwindigkeiten von 100 kBit (Standardmode), 400 kBit (Fastmode) oder 3,4 MBit (Highspeedmode) Daten austauschen. Aufgrund des einfachen elektrischen Aufbaus ist die Reichweite jedoch auf kurze Strecken (normalerweise innerhalb des ein und selben Gerätes) begrenzt.

9.1.1. Einsatzmöglichkeiten des I^2C-Bus

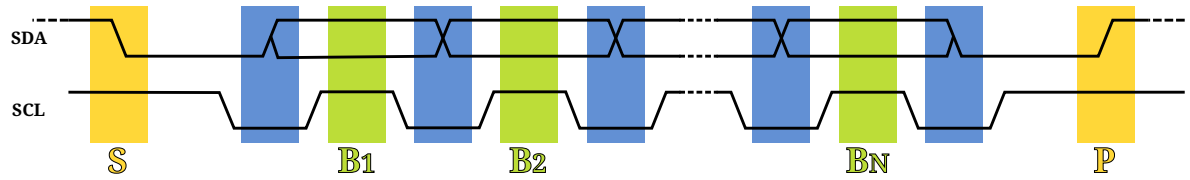
Eingesetzt wird der Bus vor allem zur Kommunikation zwischen Mikrocontrollern und externen Peripheriekomponenten. Die am Markt verfügbaren Komponenten sind sehr vielfältig. Darunter Sensoren jeder Art, Speicherbausteine, Digital-Analog-Wandler bzw. Analog-Digital-Wandler, Displays und Soundchips, Kameras, IO-Expander und vieles mehr.

Typischerweise wird der Bus immer dann eingesetzt, wenn innerhalb eines Systems mit geringem elektrischen und softwaretechnischen Aufwand Informationen ausgetauscht werden sollen. Dementsprechend ist das Busprotokoll einfach gehalten und die Abhandlung des Protokolls wird durch das eingebaute I^2C-Interface des Cortex-M4 hardwaremäßig unterstützt.



9.1.2. Physikalische Datenübertragung auf dem I²C-Bus

Jede I²C-Übertragung beginnt mit einer Startbedingung (S), welche durch eine fallende Flanke auf der Datenleitung (SDA) bei gleichzeitigem High-Pegel auf der Clock-Leitung (SCL) signalisiert wird. Der Zustand der einzelnen Datenbits (B_i) entspricht folgend dem Pegel von SDA während eines High-Pegels auf SCL. Am Ende jeder Übertragung wird eine Stoppbedingung (P) gesendet. Diese wird durch eine steigende Flanke auf SDA bei gleichzeitigem High-Pegel auf SCL signalisiert. Die Stoppbedingung signalisiert das Ende einer Übertragung und gibt den Bus für andere Teilnehmer frei.



9.1.3. BME280-Umgebungssensor

Ihre Aufgabe wird es sein, den BME280-Umgebungssensor der Firma Bosch zu integrieren, um die Temperatur, die Luftfeuchtigkeit und den Luftdruck als Fließkommawerte abrufen zu können. Ihr Mikrocontroller agiert hierbei als **Master** und der Sensor als **Slave**. Er kann theoretisch auch über das Kommunikationsprotokoll SPI gesteuert werden, wobei Sie für dieses Projekt I²C benutzen sollen.

Wie in Kapitel 4 des BME280-Datenblattes (**BME280_datenblatt.pdf** im Ressourcen-Ordner) beschrieben wird, ist es notwendig, die aus den Registern des Sensor ausgelesenen Werte auf korrekte Art und Weise zu interpretieren, zu parsen und mithilfe bestimmter vom Datenblatt mitgelieferten Berechnungen zu kompensieren. Da es relativ aufwendig wäre, dies selbstständig zu machen und von Bosch selber nicht empfohlen wird, sollen Sie die bereits fertige Bibliothek von Bosch in Betrieb nehmen. Da diese so generisch gestaltet ist, um für alle Mikroprozessoren benutzt werden zu können, muss der Benutzer im Endeffekt nur Callback-Funktionen für das Auslesen und Beschreiben von Registern über I²C (und für eine zeitliche Verzögerung) hinterlegen. Schauen Sie sich die README an, um

sich mit der Benutzung der Bibliothek vertraut zu machen.

9.1.4. Benutzung des I²C-Interfaces des Cortex-M4

Um mithilfe des Cortex-M4 mit dem Sensor interagieren zu können, müssen die dafür verwendeten beiden GPIO-Pins und die I²C-Schnittstelle selber richtig konfiguriert werden. Hier sind folgende Aspekte zu beachten:

- Da hier ein Bus betrieben werden soll, an dem mehrere Teilnehmer angeschlossen sind, die alle auch Daten senden dürfen, müssen besondere Vorkehrungen getroffen werden, um ein "Gegeneinandersenden" mehrerer Busteilnehmer zu vermeiden. Wenn beispielsweise ein Busteilnehmer versucht einen High-Pegel an den Bus anzulegen, während ein anderer Teilnehmer den Bus mit einem Low-Pegel treibt, würde ein Kurzschluss entstehen und einer der beiden Teilnehmer würde das Zeitliche segnen. Für dieses Problem gibt es jedoch eine einfache Lösung: Die Bus-Leitung wird über einen Pull-Up-Widerstand auf einem definierten High-Pegel gehalten, bis einer der Teilnehmer den Bus auf einen Low-Pegel zieht. Anschließend schaltet er seinen Ausgang auf hochohmig, wodurch der Pegel am Bus durch den Pull-Up-Widerstand wieder auf High gezogen wird. Auf diese Weise ist ein "Gegeneinandertreiben" mehrerer Busteilnehmer ausgeschlossen. Gekennzeichnet wird dieser GPIO-Modus durch die Zusatzbezeichnung **Open Drain (OD)**.
- Wie auch schon bei den Timern, möchten Sie auch hier eine alternative Funktionalität der GPIO-Pins verwenden!
- Der I²C-Standard-Bustakt beträgt 100kHz. Es können auch 400kHz für den sogenannten Fast-Mode benutzt werden, darauf verzichten wir hier allerdings.
- Es besteht die Möglichkeit, den eigenen Controller nicht nur als Master, sondern auch als Slave am I²C-Bus agieren zu lassen. Wir benötigen dies allerdings nicht und müssen somit keine slave-spezifischen Einstellungen und Modi konfigurieren.



Bearbeiten Sie nun am besten die Aufgaben in den Kapiteln **I²C-Protokoll** und **BME280-Umgebungssensor** im fünften Aufgabenblatt `abgabe5.pdf`

9.2. Kommunikation über CAN

Lernziele:

- Grundlegendes Verständnis von CAN
- Konfiguration des Cortex-M4 CAN-Interfaces
- Versenden und Empfangen von Nachrichten via CAN

Das CAN ist ein asynchrones serielles Bus-System, welches Layer 1 (physikalische Schicht)

und Layer 2 (Datensicherungsschicht) des ISO/OSI-Referenzmodells implementiert. Die maximale Anzahl an Teilnehmern hängt von den verwendeten Bustreiberbausteinen (Transceiver, physische Anschaltung an den Bus, etc.) ab. Als Übertragungsmedium finden neben Kupferleitungen auch Glasfaser Verwendung. Im Falle von Kupferleitungen arbeitet der CAN-Bus mit CAN_HIGH (CAN_H) und CAN_LOW (CAN_L) als zwei verdrillte Adern und verwendet somit eine *symmetrische Signalübertragung*. Typische Datenübertragungsraten liegen zwischen 125 kBit/s beim sog. Low-Speed CAN und 1 MBit/s beim High-Speed CAN. Es gibt aber auch noch schnellere Varianten des CAN.



Ausführliche Informationen zum *Controller Area Network (CAN)* finden sich im Web (z.B. bei Wikipedia).

9.2.1. Benutzung des CAN-Interfaces des Cortex-M4

Der eingesetzte Cortex-M4-Controller verfügt über ein sehr leistungsfähiges CAN-Interface, dessen Konfiguration relativ einfach ist.

Um das Versenden und Empfangen von CAN-Nachrichten zu vereinfachen bietet der Controller Sende- und Empfangsmailboxen an. Eine zu versendende Nachricht wird in eine der drei TX-Mailboxen kopiert (falls mindestens eine frei ist) und dort in einer Warteschlange gespeichert. Sobald ein Datenslot auf dem Bus frei wird, werden die Nachrichten versendet, wobei die Reihenfolge von der jeweiligen Nachrichtenpriorität abhängt und vom *Transmission Scheduler* bestimmt wird. Auf das CAN aufgebrachte Nachrichten sind immer Broadcasts, d.h. eine versendete Nachricht wird von allen Teilnehmern empfangen. Es gibt beim CAN auch keine Teilnehmer-Adressen; stattdessen erhält jede Nachricht einen *Identifier* (11 Bit beim *Standard Identifier* oder 29 Bit beim *Extended Identifier*), mit der ihr Inhalt zugeordnet werden kann. Da allerdings nicht alle Nachrichten für alle Netzknoten interessant sind, verfügt der Controller über 28 auf CAN1 und CAN2 aufgeteilte Bänke von Nachrichtenfiltern, die so konfiguriert werden können, dass nur noch relevante Nachrichten empfangen und in einer der beiden Empfangs-FIFOs abgelegt werden. Auf diese Art und Weise bleibt die Belastung der CPU minimal.

Die Konfiguration eines CAN-Interfaces verläuft in 4 Schritten:

- Einstellen der beiden GPIO-Leitungen im Open-Drain-Modus
- Einstellen der Interrupts (Datenempfang, Fehlerbehandlung, etc.)
- Grundeinstellung des CAN (Baudrate, Modus, etc.)
- Konfiguration der Empfangsfilter

Einstellen der Interrupts

Bei dem Aktivieren von Interrupts muss neben der Ihnen schon bekannten Konfiguration

des NVICs die Funktion `HAL_CAN_AcitvateNotification` aufgerufen werden, um auch in der CAN-Peripherie den entsprechenden Interrupt zu aktivieren.

Grundeinstellung des CAN

Die hierfür benötigte Initialisierungs-Struktur vom Typ `CAN_HandleTypeDef` bietet neben dem üblichen Feld `.Instance` (hier `CAN1` oder `CAN2` je nach benutzter CAN-Instanz des Cortex-M4) drei Arten von Feldern:

- Das Feld `.Init.Mode`, welches den Betriebsmodus des CAN-Interfaces festlegt. Neben dem Standard-Modus werden drei unterschiedliche Testmodi angeboten, welche in Kapitel 32.5 des Reference Manuals beschrieben werden.
- Die Felder `.Init.Prescaler`, `.Init.SyncJumpWidth`, `.Init.TimeSeg1` und `.Init.TimeSeg2`, welche für das sogenannte *Bit Timing* verantwortlich sind. Es ist wichtig für die zeitliche Synchronisation des Busses und zur Festlegung einer Baudrate (Anzahl an gesendeten Bits pro Sekunde). Die verschiedenen Knoten eines CAN können mit einem unterschiedlichen Takt betrieben werden, müssen über das Bit Timing allerdings eine für **alle Knoten** identische Baudrate einstellen, da ein Knoten ansonsten die Nachrichten anderer Knoten nicht richtig interpretieren kann. Ein Bit setzt sich aus den drei sich nicht überlappenden Zeitsegmenten *Synchronization segment*, *Bit segment 1* und *Bit segment 2* zusammen, wobei jedes davon ein Vielfaches vom sogenannten Zeitquantum t_q beträgt. Wir betrachten folgende Beispielkonfiguration:

  ```
  can_handle.Init.Prescaler = 21;
  can_handle.Init.SyncJumpWidth = CAN_SJW_1TQ;
  can_handle.Init.TimeSeg1 = CAN_BS1_13TQ;
  can_handle.Init.TimeSeg2 = CAN_BS2_2TQ;
  ```

  Der Prescaler bestimmt die Länge eines Zeitquantums, indem der am CAN-Interface anliegende Takt (bei uns der von APB1 mit 42MHz) durch diesen Prescaler geteilt wird. Für das obere Beispiel ergibt sich also ein Zeitquantum von $t_q = \frac{21}{42MHz} = 0.5\mu s$. Die drei Zeitsegmente ergeben sich also zu

  $$t_{SJW} = 1 * t_q$$

  $$t_{BS1} = 13 * t_q$$

  $$t_{BS2} = 2 * t_q$$

  Ein ganzes Bit hat also eine Länge von $t_B = t_{SJW} + t_{BS1} + t_{BS2} = 16 * t_q = 16 * 0.5\mu s = 8\mu s$, sodass die Baudrate $\frac{1}{8\mu s} = 125\frac{kBit}{s}$ beträgt (das entspricht dem Low-Speed CAN).

- Weitere Felder wie `.Init.TimeTriggeredMode`, `.Init.AutoBusOff`, etc., mit denen zusätzliche Modi und Funktionalitäten aktiviert werden können.

Konfiguration der Empfangsfilter

Der Cortex-M4 bietet 28 auf die beiden CAN-Instanzen (CAN1 und CAN2) verteilten Filterbänke an (die Verteilung kann konfiguriert werden), mit deren Hilfe eingehende Nachrichten ohne CPU-Belastung hardwareseitig gefiltert werden können. Jede Filterbank **i** enthält die beiden Register **CAN_FiR1** und **CAN_FiR2**. Es kann pro Filterbank zwischen den beiden Modi *Identifier List Mode* und *Mask Mode* und bei der Skalierung zwischen *16-Bit-* und *32-Bit-Skalierung* ausgewählt werden. Durch die verschiedenen Kombinationen des Modus und der Skalierung ergeben sich vier mögliche Szenarien, welche auf unterschiedliche Art und Weise die Struktur und Funktionalität der oben erwähnten Register beeinflussen (siehe Kapitel 32.7.4 und vor allem Abbildung 342 des Reference Manuals):

1. *32-Bit-Skalierung* und *Mask Mode*: Im gesamten **CAN_FiR1**-Register (alle 32 Bits) wird eine ID gespeichert mit der der Inhalt einer empfangenen Nachricht verglichen wird. Im gesamten **CAN_FiR2**-Register (alle 32 Bits) wird eine Maske gespeichert, deren Inhalt vorgibt, welche Bits der empfangenen Nachricht überhaupt mit der Filter-ID verglichen werden müssen und welche ignoriert werden können. Hier ein Beispiel:

ID (**CAN_FiR1**)	**0000 1101**	0000 **1111**	0001 0011	0000 0000
Mask (**CAN_FiR2**)	**1111 1111**	0000 **1111**	0000 0000	0000 0000
Nachricht 1	**0000 1101**	1010 **1111**	1100 1011	0000 0001
Nachricht 2	**0001 1100**	1111 **1111**	1100 1011	1100 0011

   Hier würde *Nachricht 1* den Filter passieren und empfangen werden, da alle ihre Bits, an deren Stelle die Maske eine 1 hat (im Bild fett gedruckt), mit den Bits aus der ID übereinstimmen. Nachricht 2 wiederum würde aufgrund der beiden im Bild rot markierten Bits nicht empfangen werden, da sie zwar von der Maske als relevant eingestuft werden, allerdings nicht mit den entsprechenden Bits der ID übereinstimmen!

2. *32-Bit-Skalierung* und *Identifier List Mode*: In diesem Fall fungieren beide Register (**CAN_FiR1** und **CAN_FiR2**) als IDs. Da nun keine Maske definiert ist, müssen bei einer eingehenden Nachricht **alle** Bits mit einer der beiden definierten IDs übereinstimmen. Man hat somit **zwei** Filter innerhalb einer Filterbank konfiguriert.
3. *16-Bit-Skalierung* und *Mask Mode*: Die beiden Register (**CAN_FiR1** und **CAN_FiR2**) stellen jeweils einen Filter der Filterbank dar. In den oberen 16 Bits wird jeweils die Maske und in den unteren 16 Bits die ID des entsprechenden Filters eingestellt.
4. *16-Bit-Skalierung* und *Identifier List Mode*: Hiermit kann eine Identifier Liste mit **vier** unterschiedlichen IDs definiert werden, mit denen eine empfangene Nachricht verglichen wird.

Eine Filterbank kann in der HAL mithilfe der Initialisierungs-Struktur `CAN_FilterTypeDef`

konfiguriert werden. Hierbei kann neben den relevanten Feldern `.FilterActivation`, `.FilterBank` und `.FilterFIFOAssignment` mithilfe des Felds `.FilterMode` einer der oben beschriebenen Modi festgelegt werden. `.FilterScale` legt die Filter-Skalierung fest. Die vier Felder `.FilterIdHigh`, `.FilterIdLow`, `.FilterMaskHigh` und `.FilterMaskLow` repräsentieren die oben beschriebenen Register **CAN_FiR1** und **CAN_FiR2** und müssen je nach Modus und Skalierung entsprechend benutzt werden.

Hat man die Struktur gefüllt, kann die Filterbank mithilfe von `HAL_CAN_ConfigFilter()` der CAN-Peripherie bereitgestellt werden.



Es muss immer mindestens ein Filter eingerichtet und aktiviert werden, auch wenn dieser Filter alle Nachrichten passieren lässt. Wenn gar kein Filter eingerichtet wurde, können auch keine Nachrichten empfangen werden.

Versand von Nachrichten

Ist eine der drei Sende-Mailboxen frei (überprüfbar mit der HAL-Funktion `HAL_CAN_GetTxMailboxesFreeLevel()`), so kann mithilfe der Funktion `HAL_CAN_AddTxMessage()` ein Transmission-Request ausgelöst und die Nachricht einer freien Mailbox angehängt werden. Eins der Übergabeargumente stellt ein Struct vom Typ `CAN_TxHeaderTypeDef` dar, in dem der Nachrichtenframe (bis auf die zu sendenden Daten selbst, diese werden mittels eines separaten Übergabeargumente angegeben) konfiguriert werden kann.

Empfangen von Nachrichten

Zwar ist das Empfangen von Nachrichten mittels Polling möglich, so stellt das Empfangen mittels Interrupt die elegantere Möglichkeit dar. Der Cortex-M4 bietet hierfür für die beiden Empfangs-FIFOs jeweils einen Eintrag in der Interrupt-Vektor-Tabelle. Wie auch schon bei den Timern, bietet die HAL auch hier eine für das CAN spezialisierte Interrupt-Handler-Funktion (`HAL_CAN_IRQHandler()`), die je nach ausgelöstem CAN-Interrupt eine entsprechende Callback-Funktion aufruft.

Mit `HAL_CAN_GetRxMessage()` kann hier dann die eingetroffene Nachricht abgerufen werden. Hier wird ein Zeiger auf ein Struct vom Typ `CAN_RxHeaderTypeDef` übergeben. Dieses ähnelt von der Struktur dem obigen `CAN_TxHeaderTypeDef` und unterscheidet sich hauptsächlich in dem Feld `.FilterMatchIndex`, welches nach Empfangen der Nachricht die Nummer des für das Empfangen verantwortlichen Filters (**nicht** der Filterbank!) enthält. Benutzt man pro Nachrichtenart (z.B. die Daten welches Sensors enthält die Nachricht) also einen Filter, so kann über dieses Feld eine direkte Zuweisung zur Nachrichtenart gemacht werden, ohne dass der Identifier der Nachricht überprüft werden muss.



Die Praktikumsanleitung ist hier zu Ende. Bearbeiten Sie nun die restli-

> chen Aufgaben der Kapitel **CAN-Protokoll**, **CAN-Interface des Cortex-M4** und **Wetterstation** im Aufgabenblatt `abgabe5.pdf`.

Kapitel 10. FAQ

1. Der Compiler zeigt mir Unmengen an Fehlermeldungen an, die scheinbar in Zusammenhang mit der STM32Cube HAL stehen. Was kann ich tun?

   Lösung: Prüfen Sie, ob Sie in das entsprechende Programmmodul die zentrale Header-Datei der ST-Lib mittels *#include "stm32f4xx.h* inkludiert haben. Meist empfielt es sich diese Datei bereits in der *.h-Datei Ihres Moduls zu inkludieren.

2. Ich möchte eine Peripheriekomponente benutzen. Das Programm lässt sich einwandfrei kompilieren und linken, funktioniert jedoch nicht.

   Lösung: Gehen Sie aufmerksam folgende Fehler-Checkliste durch:

   a. Wurden alle benötigten Takte eingestellt?
   b. Wurden die benötigten GPIOs (richtig) konfiguriert?
   c. Wurden die benötigten externen Interrupts (richtig) konfiguriert?
   d. Wurde der NVIC (richtig) konfiguriert?
   e. Haben die ISRs den richtigen Namen?
   f. Wird das Pending Bit innerhalb der ISR gelöscht?
   g. Werden in der Main Funktion alle Initialisierungen ausgeführt?
   h. Wurde die benötigte Peripherie (richtig) konfiguriert?

3. Ich möchte ein von außen konfigurierbares ADC-Modul implementieren, bei dem Channel und Rank in der Scanning-Sequencer Liste frei wählbar sind. Was passiert, wenn der Benutzer versehentlich 2 Channels den gleichen Rank zu weist?

   Lösung: Die Rang-Liste des ADC-Sequenzers stellt eine Art Tabelle mit max. 16 einträgen dar. Durch die Konfiguration eines regulären ADC-Channels mit Rank *n* wird in Zeile *n* der Sequenzer-Tabelle die jeweilige Channel-Nummer eingetragen. Sollte anschließend ein neuer Channel auf diesem Rank konfiguriert werden, so wird der Tabelleneintrag überschrieben. Gültig ist hier also immer die zuletzt gemachte Konfiguration auf Rank *n*.

4. Ich habe meinen Programmcode verändert, auf dem Controller scheint aber immernoch das alte Programm abzulaufen.

   Lösung: Denken Sie daran, dass Sie ihr Projekt zunächst kompilieren und es anschließend auf den Controller überspielen. Vergessen Sie auch nicht Ihre Datei zu speichern.

5. Beim Debugging bekomme ich die Fehlermeldung "Could not determine GDB version ...

", wenn ich mir die ausführlichen Informationen ausklappe.

Lösung: Das kann mit den NCurses Bibliotheken zusammenhängen (insbesondere bei einem Arch Linux Abkömmling). Hier sind zwei Links, die Abhilfe schaffen können:

a. Kompatibilitäts-Bibliotheken (https://aur.archlinux.org/packages/ncurses5-compat-libs/)
b. Fertig konfigurierte IDE (https://aur.archlinux.org/packages/stm32cubeide/)

6. Ich programmiere meine 8-Segment Anzeige aber es leuchtet keine LED.

Lösung: Es kann sein, dass Ihre Steckverbindung einen Wackelkontakt hat. Wackeln Sie ein wenig an der Anzeige oder ziehen Sie auch minimal heraus.

Kapitel 11. Literatur

[1] ST Microelectronics, “User Manual - Discovery kit with STM32F429ZI MCU.” discovery-kit_user-manual.pdf.

[2] WaveShare, “Open429Z-D User Manual.” Open429Z-D-UserManual.pdf.

[3] ST Microelectronics, “Reference Manual - STM32F405xx/07xx, STM32F415xx/17xx, STM32F42xxx and STM32F43xxx advanced ARM-based 32-bit MCUs.” STM32F4-Reference-Manual.pdf.

[4] WaveShare, “Open429Z-D Schaltplan.” Open429Z-D-Schematic.pdf.

[5] WaveShare, “8-Segment LED Adapter Board Schaltplan.” 8-Seg-LED-Board-Schematic.pdf.

[6] WaveShare, “BME280 Umgebungssensor Adapter Board Schaltplan.” BME280-Environmental-Sensor-Schematic.pdf.

[7] WaveShare, “Analog Test Board Schaltplan.” Analog-Test-Board-Schematic.pdf.

[8] ST Microelectronics, “STM32F429XX Datasheet.” STM32F429-Datasheet.pdf.

[9] Bosch Sensortec GmbH, “BME280 Datasheet.” BME280_datenblatt.pdf.

[10] ST Microelectronics, “L3FSD20 Datasheet.” L3GD20_datenblatt.pdf.

[11] H. Pfaffelmoser, “Einführung in die ARM CortexTM-M3 Mikrocontroller - Progammierung durch eine exemplarische Anwendungsimplementierung unter Einbindung eines Echtzeitbetriebssystems.” HPFAFF_Diplomarbeit_Hauke-Pfaffelmoser.pdf, 2012.

[12] ST Microelectronics, “Programming manual - STM32 Cortex ® -M4 MCUs and MPUs programming manual.” STM32_Cortex-M4_MCUs_and_MPUs_programming_manual.pdf.